

LAPORAN SISTEM PERKIRAAN CUACA MENGGUNAKAN SENSOR TEMPERATUR & HUMIDITY



NAMA ANGGOTA KELOMPOK:

Faradila Fasya	(20230140151)
Regina Rana Nabila	(20230140154)
Aini Rana Salsabil	(20230140178)
Adilla Roisa Santosa	(20230140179)
Putri Aulia Syafira Arif	(20230140194)

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH YOGYAKARTA

2026

DAFTAR ISI

BAB I – PENDAHULUAN	2
1.1 Latar Belakang	2
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	3
1.4 Manfaat Penelitian	3
1.5 Batasan Masalah	3
BAB II – DIAGRAM UML	3
2.1 Use Case	4
A. Deskripsi Aktor	6
B. Deskripsi Use Case Utama	6
C. Penjelasan Relasi Diagram	7
2.2 Flowchart	7
A. Tahap Autentikasi dan Akses	10
B. Percabangan Peran (Role-Based Access Control)	10
2.3 Activity Diagram	10
2.3.1 Activity Diagram Register	10
A. Deskripsi Alur Kerja	12
B. Simbol-Simbol Utama	12
2.3.2 Activity Diagram Login	13
A. Deskripsi Alur Kerja	14
B. Simbol-Simbol Utama	15
2.3.3 Activity Diagram Lihat Cuaca Negara/Kota (User)	15
A. Deskripsi Alur Kerja	16
B. Simbol-Simbol Utama	16

2.3.4 Activity Diagram Lihat Forecast Cuaca 5 Hari (Admin)	17
A. Deskripsi Alur Kerja.....	18
B. Simbol Utama pada Diagram	18
2.3.5 Activity Diagram Kelola User (Admin)	18
A. Deskripsi Alur Kerja.....	20
2.3.6 Activity Diagram Lihat Log Sensor (Admin)	21
A. Deskripsi Alur Kerja.....	22
B. Simbol Utama pada Diagram	22
2.3.7 Activity Diagram Kirim Data Sensor (IOT)	23
A. Deskripsi Alur Kerja.....	24
B. Simbol Utama pada Diagram	24
2.4 Sequence Diagram.....	24
2.4.1 Sequence Diagram (User)	25
A. Deskripsi Alur Interaksi	26
B. Komponen Teknis dalam Diagram	27
2.4.2 Sequence Diagram (Admin).....	27
2.4.3 Sequence Diagram (IOT)	31
A. Deskripsi Alur Interaksi	32
B. Komponen Arsitektur dalam Diagram	32
2.5 Class Diagram	33
A. Deskripsi Komponen Kelas.....	37
B. Hubungan Antar Kelas (Relationships).....	37
2.6 Entity Relationship Diagram (ERD)	38
A. Deskripsi Entitas dan Atribut	40
B. Relasi Antar Entitas	40
2.7 Arsitektur Diagram	40
A. Lapisan Arsitektur (Layers)	43
B. Alur Kerja Komunikasi Data	43

BAB III – APLIKASI	44
3.1 Deskripsi Alat dan Implementasi Perangkat Keras	44
3.1.1 Perangkat Keras yang Digunakan.....	44
1. ESP32.....	44
2. Sensor Temperatur dan Humidity (DHT11/DHT22)	44
3. Kabel jumper	45
4. Breadboard	46
5. Power supply / kabel USB	46
3.1.2 Alur Kerja Perangkat Keras.....	47
3.1.3 Foto Alat.....	47
3.2 Tampilan Aplikasi Web	48
3.2.1 Register User	48
3.2.2 Login User	49
3.2.3 Homepage User	50
3.2.4 Data Sensor User	51
3.2.5 Register Admin.....	52
3.2.6 Login Admin	53
3.2.7 Manage Data User untuk Admin	54
3.3 Struktur Folder dan Source Code Frontend	57
3.3.1 File dan Folder Root Frontend	57
3.3.2 Folder public	58
3.3.3 Folder src	59
3.3.4 Folder src/components.....	59
3.4 Struktur Folder dan Source Code Backend.....	60
3.4.1 File dan Folder Root Backend	60
3.4.2 Folder config	61
3.4.3 Folder controllers.....	61
3.4.4 Folder middleware.....	61

3.4.5 Folder models	61
3.4.6 Folder routes.....	62
3.5 Alur Kerja Sistem	62
BAB IV – KESIMPULAN	62
DAFTAR PUSTAKA	63

BAB I – PENDAHULUAN

1.1 Latar Belakang

Cuaca merupakan salah satu faktor penting yang memengaruhi berbagai aktivitas manusia, seperti kegiatan sehari-hari, pertanian, transportasi, hingga perencanaan kegiatan luar ruangan. Kondisi cuaca yang berubah-ubah dapat menimbulkan berbagai dampak, baik positif maupun negatif, terutama apabila tidak diantisipasi dengan baik. Oleh karena itu, informasi cuaca yang akurat dan mudah diakses menjadi kebutuhan penting bagi masyarakat dalam mendukung pengambilan keputusan sehari-hari.(Badan Meteorologi, 2023)

Perkembangan teknologi informasi saat ini, khususnya dalam bidang *Internet of Things* (IoT), memungkinkan proses pengambilan data lingkungan dilakukan secara otomatis dan real-time. Teknologi IoT memanfaatkan perangkat sensor untuk mengumpulkan data fisik dari lingkungan sekitar dan mengirimkannya ke sistem untuk diolah menjadi informasi yang bermanfaat (Ashton, 2009). Sensor temperatur dan kelembapan udara (humidity) merupakan salah satu jenis sensor yang umum digunakan untuk memantau kondisi lingkungan dan dapat dijadikan dasar dalam melakukan perkiraan cuaca sederhana(Satria, 2022).

Berdasarkan permasalahan tersebut, dikembangkan sebuah Sistem Perkiraan Cuaca Menggunakan Sensor Temperatur dan Humidity yang mampu membaca data suhu dan kelembapan udara secara berkala, kemudian mengolah data tersebut untuk menghasilkan informasi perkiraan kondisi cuaca. Sistem ini diharapkan dapat membantu pengguna dalam memperoleh informasi cuaca secara cepat dan efisien, serta menjadi bentuk penerapan teknologi IoT dalam pemantauan lingkungan.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Bagaimana cara merancang sistem perkiraan cuaca menggunakan sensor temperatur dan humidity?
2. Bagaimana proses pengambilan dan pengolahan data suhu serta kelembapan udara?
3. Bagaimana sistem dapat menampilkan informasi cuaca secara jelas dan mudah dipahami oleh pengguna?

1.3 Tujuan Penelitian

Tujuan dari pembuatan sistem perkiraan cuaca ini adalah:

1. Merancang dan membangun sistem perkiraan cuaca berbasis sensor temperatur dan humidity.
2. Mengimplementasikan pembacaan data suhu dan kelembapan secara real-time.
3. Menyajikan informasi perkiraan cuaca yang sederhana dan informatif bagi pengguna.

1.4 Manfaat Penelitian

Adapun manfaat dari pembuatan sistem ini antara lain:

1. Membantu pengguna dalam mengetahui kondisi cuaca di suatu lingkungan.
2. Menjadi media pembelajaran bagi mahasiswa dalam memahami penerapan sensor dan IoT.
3. Memberikan gambaran penerapan teknologi sederhana untuk monitoring lingkungan.

1.5 Batasan Masalah

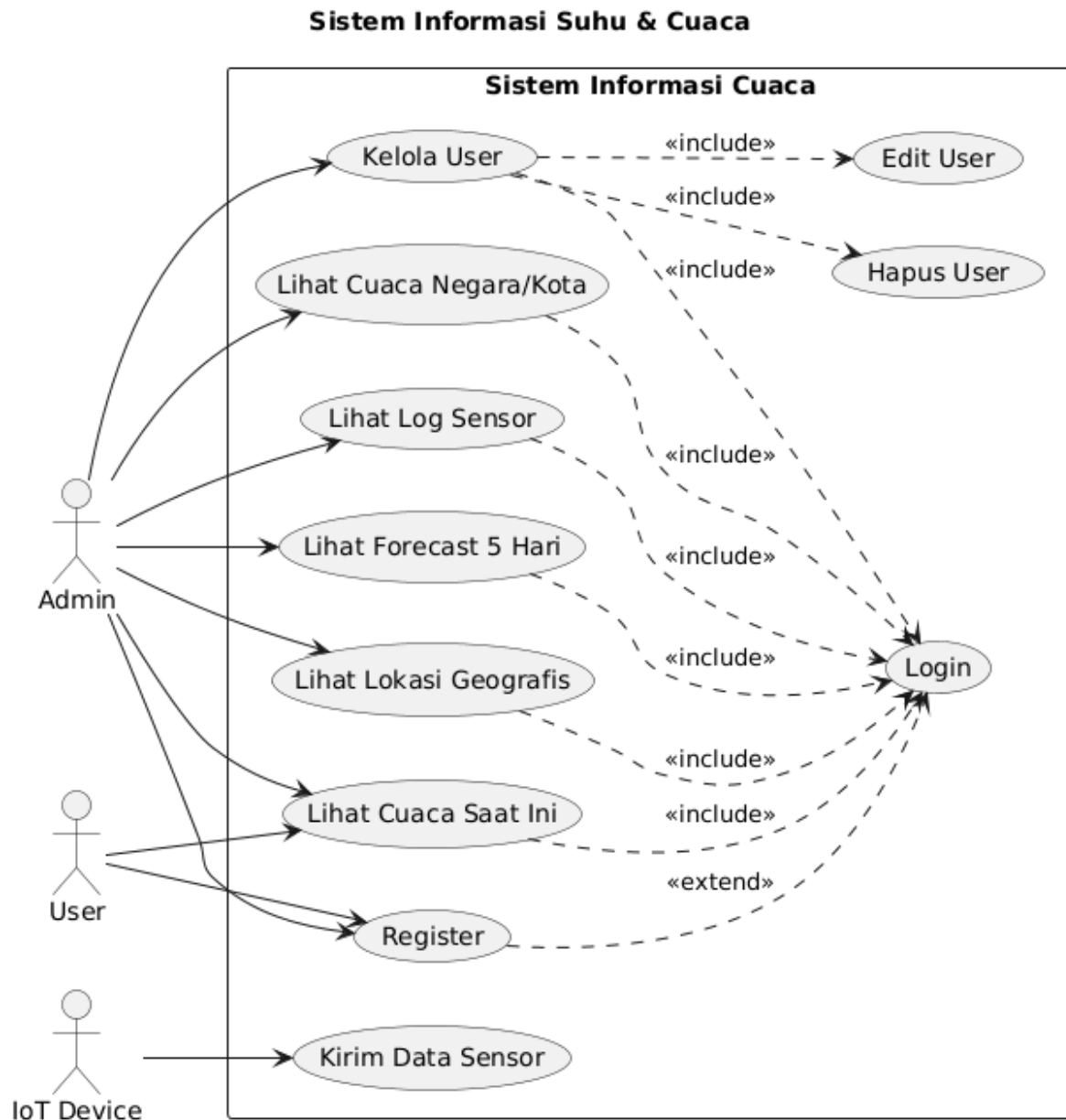
Agar pembahasan tidak meluas, maka sistem ini memiliki batasan masalah sebagai berikut:

1. Sistem hanya menggunakan sensor temperatur dan humidity sebagai parameter cuaca.
2. Perkiraan cuaca yang dihasilkan bersifat sederhana dan tidak menggantikan sistem prakiraan cuaca resmi.
3. Sistem diuji dalam skala lingkungan terbatas.

BAB II – DIAGRAM UML

2.1 Use Case

Use Case Diagram ini menggambarkan hubungan interaksi antara aktor dan fungsi sistem dalam Sistem Informasi Suhu dan Cuaca.



Gambar 1 Use Case Diagram Sistem Informasi Suhu dan Cuaca

@startuml

title Use Case Diagram

title Sistem Informasi Suhu & Cuaca

left to right direction

actor Admin

actor User

actor "IoT Device" as IoT

rectangle "Sistem Informasi Cuaca" {

 usecase "Login" as UC_Login

 usecase "Register" as UC_Register

 usecase "Lihat Cuaca Saat Ini" as UC_Current

 usecase "Lihat Cuaca Negara/Kota" as UC_Country

 usecase "Lihat Forecast 5 Hari" as UC_Forecast

 usecase "Lihat Lokasi Geografis" as UC_Map

 usecase "Lihat Log Sensor" as UC_Log

 usecase "Kelola User" as UC_ManageUser

 usecase "Edit User" as UC_EditUser

 usecase "Hapus User" as UC_DeleteUser

 usecase "Kirim Data Sensor" as UC_Send

}

' ===== RELASI AKTOR =====

Admin --> UC_Current

Admin --> UC_Country

Admin --> UC_Forecast

Admin --> UC_Map

Admin --> UC_Log

Admin --> UC_ManageUser

Admin --> UC_Register

User --> UC_Current

User --> UC_Register

IoT --> UC_Send

'===== SEMUA AKSES USER & ADMIN WAJIB LOGIN =====

UC_Current ..> UC_Login : <<include>>

UC_Country ..> UC_Login : <<include>>

UC_Forecast ..> UC_Login : <<include>>

UC_Map ..> UC_Login : <<include>>

UC_Log ..> UC_Login : <<include>>

UC_ManageUser ..> UC_Login : <<include>>

UC_Register ..> UC_Login : <<extend>>

'===== DETAIL KELOLA USER =====

UC_ManageUser ..> UC_EditUser : <<include>>

UC_ManageUser ..> UC_DeleteUser : <<include>>

@enduml

A. Deskripsi Aktor

Terdapat tiga aktor utama yang berinteraksi dengan sistem:

1. Admin: Pengguna dengan hak akses tertinggi untuk memantau data dan mengelola pengguna.
2. User: Pengguna umum yang dapat melihat informasi cuaca dan melakukan pendaftaran akun.
3. IoT Device: Perangkat keras yang mengirimkan data sensor ke sistem secara otomatis.

B. Deskripsi Use Case Utama

Fungsionalitas sistem dibagi menjadi beberapa kategori utama:

1. Manajemen Akun: Meliputi proses *Login*, *Register*, dan *Kelola User* (khusus Admin).
2. Monitoring Cuaca & Data: Menampilkan kondisi terkini, *forecast* 5 hari, serta lokasi geografis.
3. Input Data: Proses pengiriman data suhu oleh *IoT Device*.

C. Penjelasan Relasi Diagram

1. Relasi <<include>> ke Login: Menunjukkan bahwa fitur monitoring dan manajemen wajib melalui proses autentikasi.
2. Relasi <<include>> pada Kelola User: Saat Admin memilih menu "Kelola User", maka fungsi "Edit" dan "Hapus" menjadi bagian yang tidak terpisahkan dari manajemen tersebut.
3. Relasi <<extend>> pada Register: Menunjukkan bahwa pendaftaran akun adalah fitur tambahan yang opsional bagi pengguna sebelum masuk ke sistem.

2.2 Flowchart

Flowchart ini menggambarkan alur logika dan urutan proses yang dijalankan oleh aplikasi dari awal hingga akhir penggunaan sistem.



Gambar 2 Flowchart Diagram Sistem Informasi Suhu dan Cuaca

@startuml

title Flowchart Sistem Informasi Suhu & Cuaca Berbasis Web

start

:User mengakses website;

if (Sudah login?) then (Ya)

```
else (Tidak)

:Tampilkan halaman Login / Register;

if (Pilih Register?) then (Ya)

:Input data registrasi;

:Simpan data user;

endif

:Input username & password;

:Validasi login;

endif

if (Role = Admin?) then (Admin)

:Masuk Dashboard Admin;

fork

:Monitoring suhu sensor;

:Lihat riwayat log sensor;

fork again

:Cari negara & kota;

:Lihat cuaca saat ini;

:Lihat prakiraan cuaca 5 hari;

:Lihat lokasi geografis;

fork again

:Kelola user (CRUD);

end fork

else (User)

:Masuk Dashboard User;

:Cari negara & kota;
```

```
:Lihat cuaca saat ini;  
endif  
:Logout;  
stop  
@enduml
```

A. Tahap Autentikasi dan Akses

Proses dimulai ketika pengguna mengakses website, di mana sistem melakukan pengecekan status:

1. Pengecekan Login: Jika pengguna sudah login, sistem langsung mengarahkan ke pengecekan peran (role).
2. Registrasi: Jika belum memiliki akun, pengguna diarahkan ke halaman Register.
3. Login: Pengguna memasukkan kredensial untuk divalidasi oleh sistem.

B. Percabangan Peran (Role-Based Access Control)

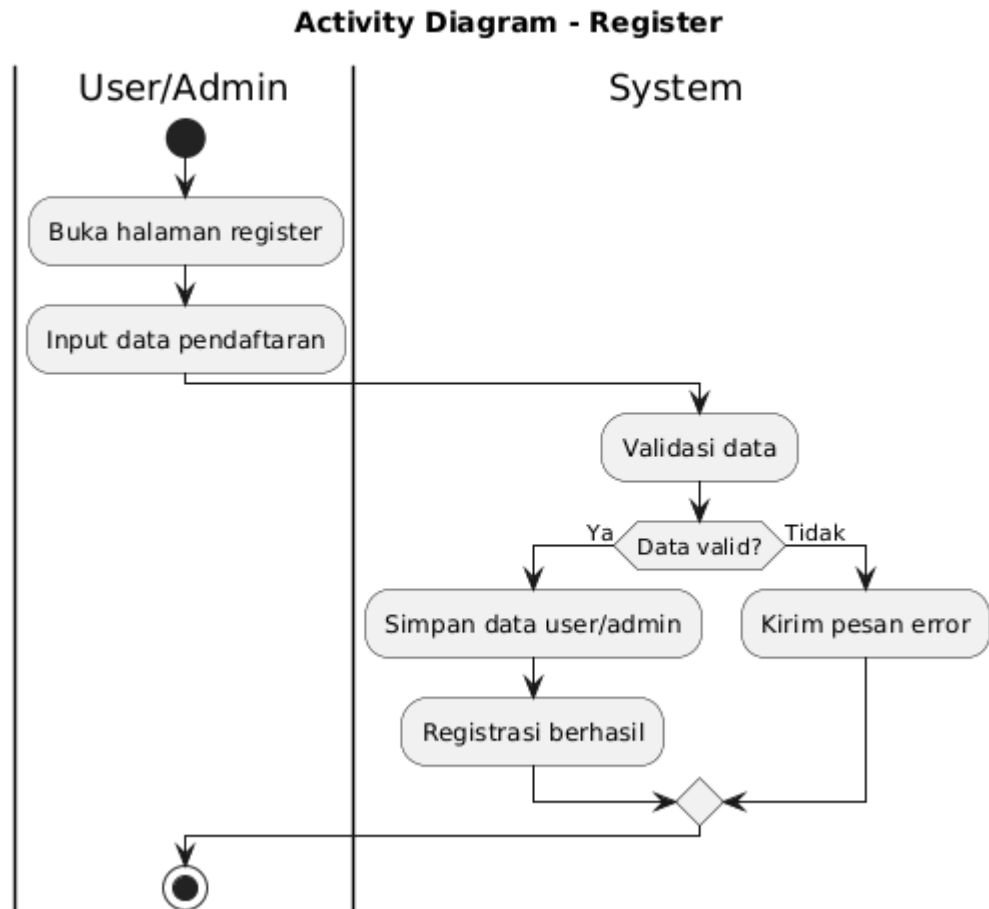
Setelah validasi berhasil, sistem membagi hak akses berdasarkan tingkatan pengguna:

1. Alur Admin: Memiliki akses penuh ke fitur manajemen sensor (real-time & log), informasi cuaca lengkap (5 hari & peta), serta manajemen pengguna (CRUD).
2. Alur User: Memiliki akses terbatas untuk melakukan pencarian lokasi dan melihat kondisi cuaca saat ini.
3. Akhir Proses Sesi berakhir ketika pengguna memilih opsi Logout, yang kemudian akan memutus akses ke dashboard dan kembali ke halaman awal.

2.3 Activity Diagram

2.3.1 Activity Diagram Register

Activity Diagram ini menggambarkan alur proses pendaftaran akun baru dalam Sistem Informasi Suhu dan Cuaca yang melibatkan interaksi antara aktor (User dan Admin) dengan sistem.



Gambar 3 Activity Diagram Register

@startuml

title Activity Diagram - Register

|User/Admin|

start

:Buka halaman register;

:Input data pendaftaran;

|System|

:Validasi data;

if (Data valid?) then (Ya)

```
:Simpan data user/admin;  
:Registrasi berhasil;  
else (Tidak)  
:Kirim pesan error;  
endif  
|User/Admin|  
stop  
@enduml
```

A. Deskripsi Alur Kerja

Proses pendaftaran dibagi menjadi dua kolom otoritas (*swimlane*) untuk memperjelas tanggung jawab masing-masing entitas:

1. Sisi User/Admin:
 - a. Mulai: Pengguna memulai proses dengan mengakses halaman registrasi pada aplikasi web.
 - b. Input Data: Pengguna mengisi formulir pendaftaran yang diperlukan, meliputi nama, *email*, *username*, dan *password*.
2. Sisi Sistem:
 - a. Validasi Data: Setelah data dikirim, sistem melakukan pengecekan (*validasi*) untuk memastikan format data sudah benar dan *username* belum digunakan oleh pengguna lain.
 - b. Percabangan (*Decision*):
 - 1) Data Valid: Sistem menyimpan informasi pengguna ke dalam *database* dan menampilkan pesan bahwa registrasi berhasil.
 - 2) Data Tidak Valid: Sistem membatalkan proses penyimpanan dan mengirimkan pesan *error* (peringatan) agar pengguna memperbaiki data yang salah.

B. Simbol-Simbol Utama

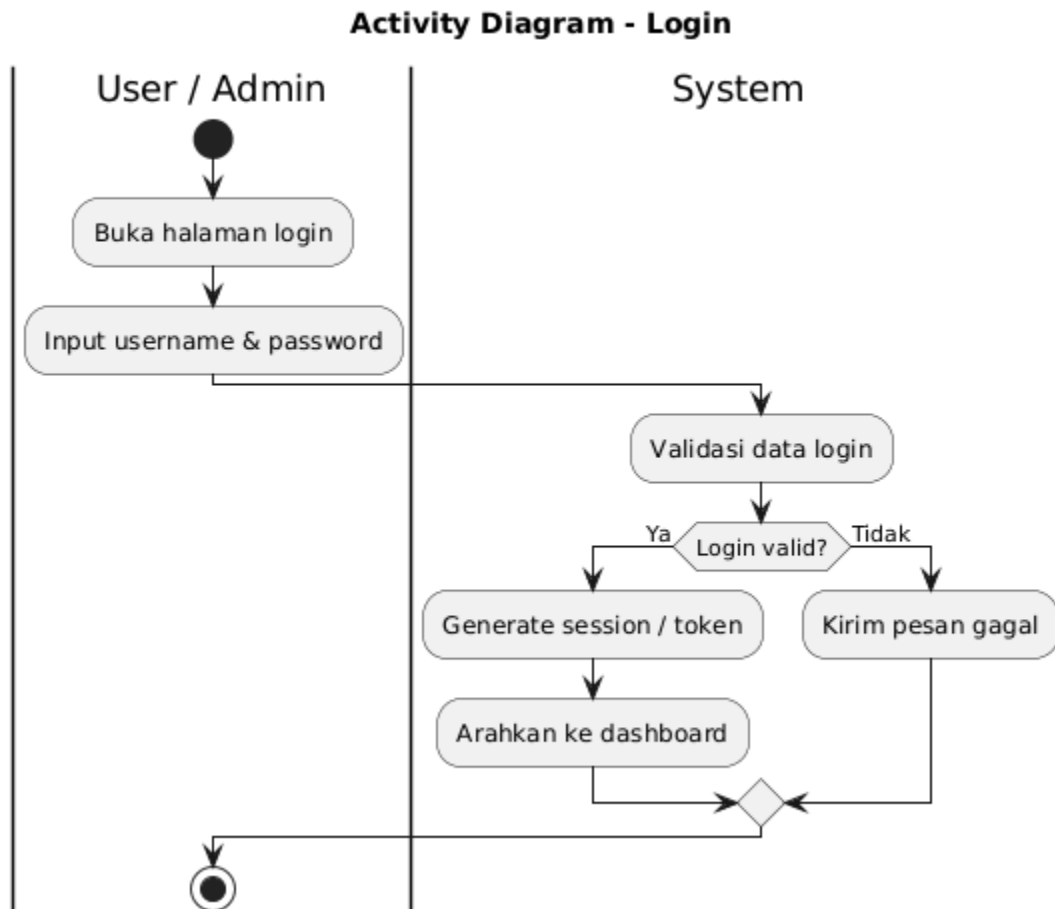
Untuk mempermudah pembacaan diagram, berikut adalah penjelasan simbol-simbol yang digunakan:

1. *Initial Node* (Lingkaran Hitam): Menandakan titik awal dimulainya proses registrasi.

2. *Swimlane*: Garis vertikal yang memisahkan tanggung jawab antara tindakan manual pengguna (*User/Admin*) dan respon otomatis aplikasi (*System*).
3. *Decision* (Belah Ketupat): Titik logika di mana sistem menentukan apakah data memenuhi syarat untuk disimpan atau harus dikembalikan ke tahap input.
4. *Final Node* (Lingkaran dengan Bingkai): Menandakan titik akhir dari seluruh rangkaian proses registrasi.

2.3.2 Activity Diagram Login

Activity Diagram ini menggambarkan proses autentikasi pengguna pada Sistem Informasi Suhu dan Cuaca untuk memastikan bahwa hanya pengguna terdaftar yang dapat mengakses fitur-fitur pada dashboard sistem.



Gambar 4 Activity Diagram Login

@startuml

title Activity Diagram - Login

|User / Admin|

start

:Buka halaman login;

:Input username & password;

|System|

:Validasi data login;

if (Login valid?) then (Ya)

 :Generate session / token;

 :Arahkan ke dashboard;

else (Tidak)

 :Kirim pesan gagal;

endif

|User / Admin|

stop

@enduml

A. Deskripsi Alur Kerja

Proses ini terbagi ke dalam dua entitas utama dengan urutan langkah sebagai berikut:

1. Sisi User / Admin:
 - a. Akses Halaman: Pengguna memulai aktivitas dengan membuka halaman *login* pada aplikasi.
 - b. Input Kredensial: Pengguna memasukkan *username* dan *password* yang telah didaftarkan sebelumnya.
2. Sisi Sistem:
 - a. Validasi Data: Sistem menerima input dan melakukan pencocokan data dengan *database* pengguna.
 - b. Percabangan (*Login Valid?*):
 - 1) Ya: Jika data sesuai, sistem akan mengarahkan pengguna ke halaman *dashboard* sesuai perannya (*Admin* atau *User*).

- 2) Tidak: Jika data tidak ditemukan atau *password* salah, sistem mengirimkan pesan peringatan agar pengguna mencoba kembali.

B. Simbol-Simbol Utama

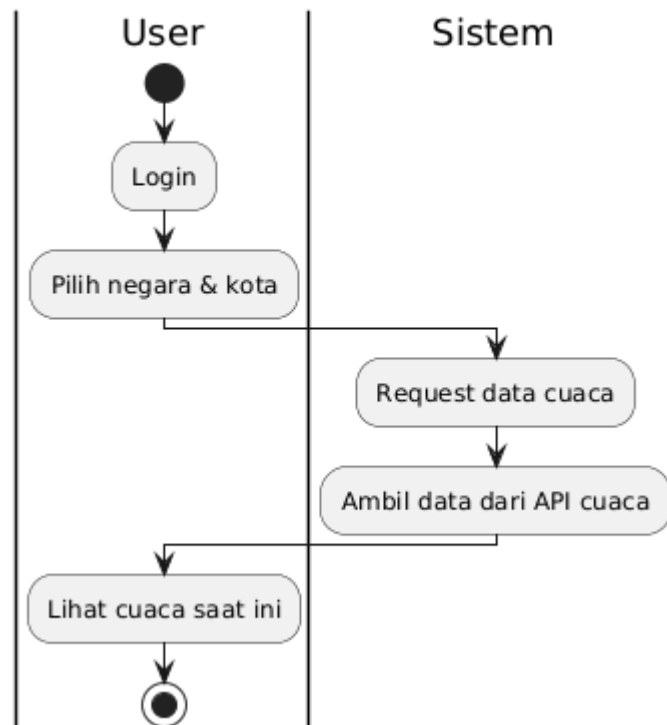
Penjelasan simbol yang digunakan dalam diagram ini meliputi:

1. *Initial Node*: Menunjukkan titik awal proses ketika pengguna berada di halaman *login*.
2. *Decision Node*: Titik krusial di mana sistem menentukan keberhasilan akses berdasarkan kecocokan data.
3. *Merge Node*: Titik pertemuan alur kembali sebelum proses berakhir, baik pada kondisi berhasil maupun gagal.
4. *Final Node*: Menandakan selesainya rangkaian proses autentikasi.

2.3.3 Activity Diagram Lihat Cuaca Negara/Kota (User)

Activity Diagram ini menggambarkan proses perolehan informasi cuaca berdasarkan lokasi yang dipilih oleh pengguna melalui interaksi antara pengguna dan sistem yang terintegrasi dengan layanan eksternal (API).

Activity Diagram - Lihat Cuaca Negara/Kota



Gambar 5 Activity Diagram Lihat Cuaca Negara/Kota (User)

@startuml

title Activity Diagram - Lihat Cuaca Negara/Kota

|User|

start

:Login;

:Pilih negara & kota;

|Sistem|

:Request data cuaca;

:Ambil data dari API cuaca;

|User|

:Lihat cuaca saat ini;

stop

@enduml

A. Deskripsi Alur Kerja

Proses ini dibagi menjadi dua kolom tanggung jawab (*swimlane*):

1. Sisi User:

- a. Autentikasi: Pengguna harus melakukan Login terlebih dahulu untuk dapat mengakses fitur pencarian cuaca.
- b. Input Lokasi: Pengguna memilih atau memasukkan nama negara dan kota yang ingin diketahui informasi cuacanya.
- c. Output: Setelah sistem memproses permintaan, pengguna dapat melihat kondisi cuaca saat ini pada layar perangkat mereka.

2. Sisi Sistem:

- a. Permintaan Data: Sistem menerima input dari pengguna dan melakukan Request data cuaca.
- b. Integrasi Eksternal: Sistem melakukan pengambilan data dari API cuaca (seperti OpenWeatherMap atau penyedia layanan serupa) untuk mendapatkan informasi suhu dan cuaca yang akurat serta *real-time*.

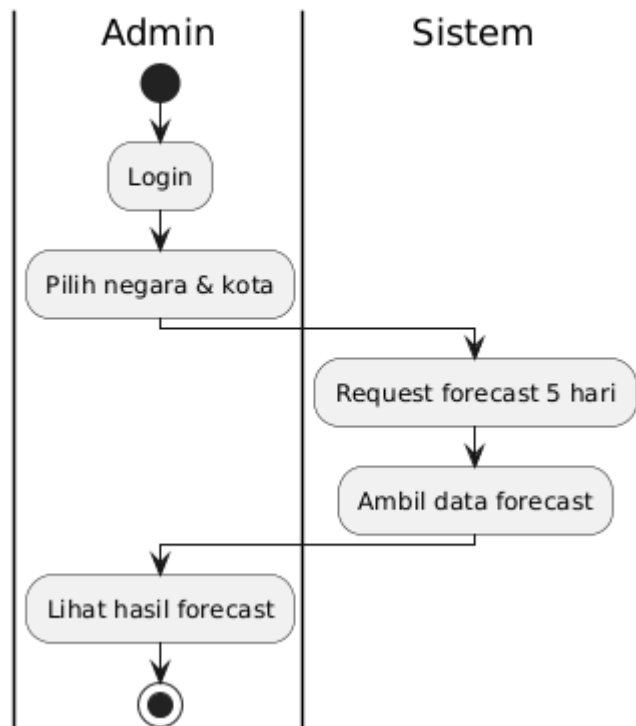
B. Simbol-Simbol Utama

1. Initial Node (Lingkaran Hitam): Menandakan dimulainya aktivitas ketika pengguna membuka menu pencarian cuaca.
2. Swimlane (User & Sistem): Membedakan aktivitas yang dilakukan oleh manusia dengan aktivitas otomatis yang dilakukan oleh perangkat lunak.
3. Final Node (Lingkaran dengan Bingkai): Menandakan bahwa proses pengambilan informasi cuaca telah selesai setelah data ditampilkan kepada pengguna.

2.3.4 Activity Diagram Lihat Forecast Cuaca 5 Hari (Admin)

Activity Diagram ini menggambarkan proses teknis yang dilakukan oleh aktor Admin untuk memperoleh prediksi cuaca jangka menengah (lima hari ke depan). Berbeda dengan fitur cuaca saat ini, proses ini melibatkan pengolahan data prakiraan yang lebih mendalam pada sisi sistem.

Activity Diagram - Lihat Forecast Cuaca 5 Hari



Gambar 6 Activity Diagram Lihat Forecast Cuaca 5 Hari (Admin)

@startuml

title Activity Diagram - Lihat Forecast Cuaca 5 Hari

|Admin|

start

:Login;

:Pilih negara & kota;

|Sistem|

:Request forecast 5 hari;

:Ambil data forecast;

|Admin|

:Lihat hasil forecast;

stop

@enduml

A. Deskripsi Alur Kerja

Proses ini melibatkan interaksi terstruktur antara Admin dan Sistem:

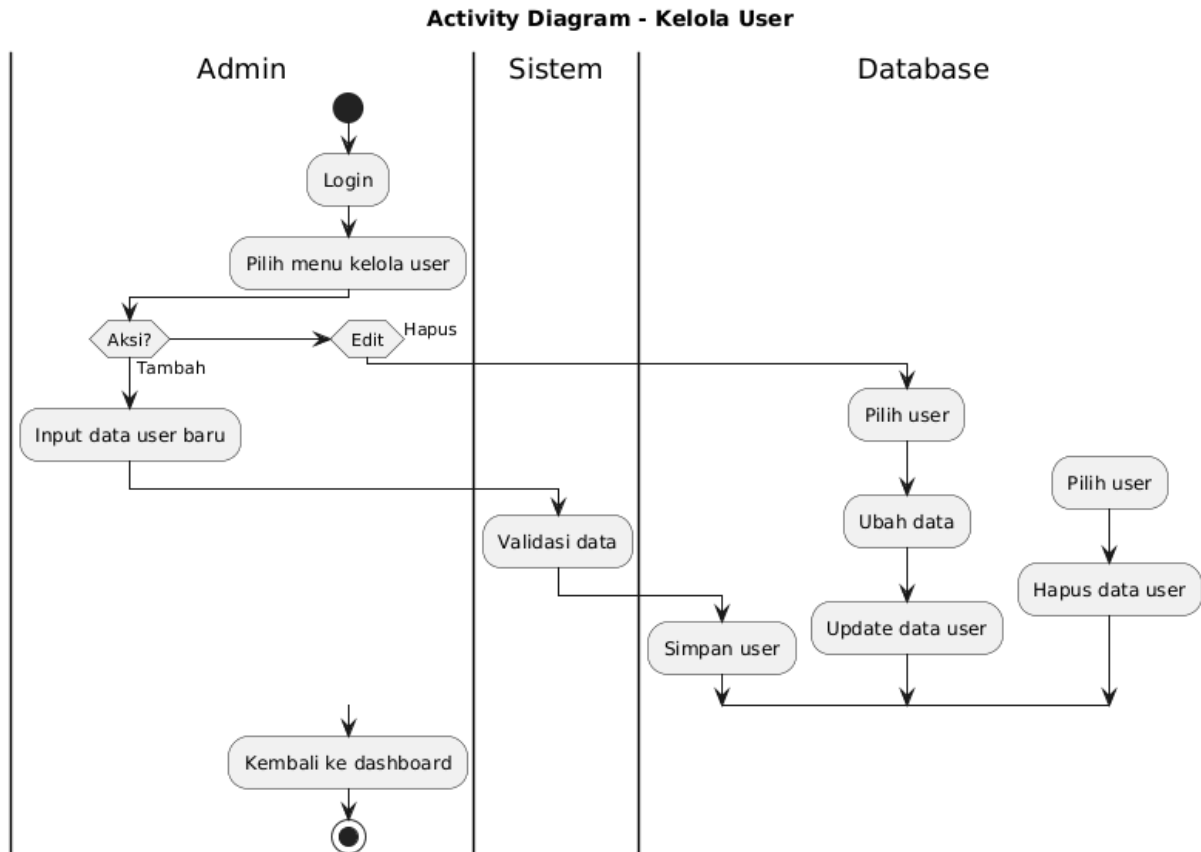
1. Sisi Admin:
 - a. Login: Sebagai prasyarat akses, Admin harus melakukan autentikasi ke dalam sistem.
 - b. Pemilihan Lokasi: Admin menentukan wilayah dengan memilih negara dan kota yang spesifik pada antarmuka dashboard.
 - c. Output: Setelah proses pengambilan data selesai, Admin dapat melihat hasil forecast yang menyajikan prediksi cuaca untuk 5 hari mendatang.
2. Sisi Sistem:
 - a. Request Forecast: Sistem menerima parameter lokasi dari Admin dan melakukan permintaan khusus (request forecast 5 hari) ke layanan penyedia data.
 - b. Pengambilan Data: Sistem secara otomatis melakukan proses ambil data forecast untuk diterjemahkan menjadi informasi yang mudah dipahami oleh pengguna.

B. Simbol Utama pada Diagram

- **Initial Node:** Menandakan dimulainya aktivitas di dashboard Admin.
- **Swimlane (Admin & Sistem):** Memisahkan tanggung jawab antara input manual yang dilakukan Admin dan pemrosesan data otomatis oleh Sistem.
- **Final Node:** Menandakan akhir dari alur aktivitas setelah informasi berhasil ditampilkan.

2.3.5 Activity Diagram Kelola User (Admin)

Activity Diagram ini menggambarkan prosedur fungsional yang dilakukan oleh aktor Admin dalam mengelola data pengguna melalui operasi CRUD (Create, Read, Update, Delete). Proses ini memastikan bahwa Admin memiliki kendali penuh terhadap pengelolaan dan aksesibilitas pengguna di dalam sistem.



Gambar 7 Activity Diagram Kelola User (Admin)

@startuml

title Activity Diagram - Kelola User

|Admin|

start

:Login;

:Pilih menu kelola user;

if (Aksi?) then (Tambah)

```

:Input data user baru;

|Sistem|

:Validasi data;

|Database|

:Simpan user;

elseif (Edit)

:Pilih user;

:Ubah data;

|Database|

:Update data user;

else (Hapus)

:Pilih user;

|Database|

:Hapus data user;

endif

|Admin|

:Kembali ke dashboard;

stop

@enduml

```

A. Deskripsi Alur Kerja

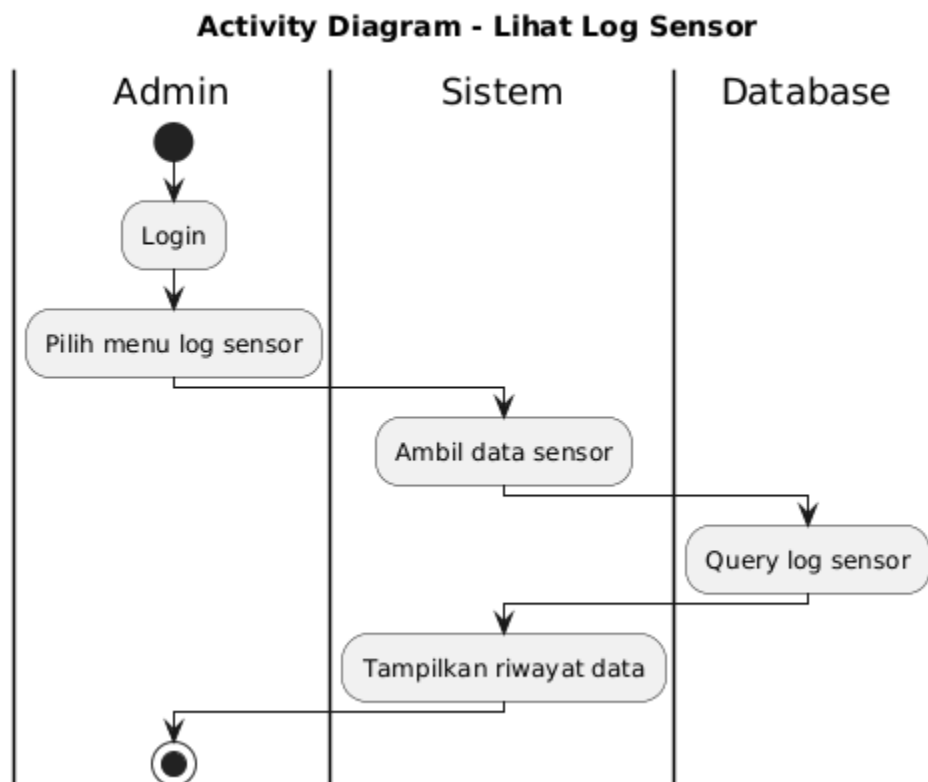
Proses manajemen user ini melibatkan interaksi antara Admin, Sistem, dan Database:

1. Sisi Admin:
 - a. Autentikasi: Admin harus melakukan Login terlebih dahulu untuk mengakses fitur administratif.
 - b. Pemilihan Aksi: Setelah masuk ke menu Kelola User, Admin dapat memilih salah satu dari tiga tindakan utama:
 - 1) Tambah: Admin menginput data user baru.

- 2) Edit: Admin memilih user yang ada dan mengubah informasinya.
 - 3) Hapus: Admin memilih user tertentu untuk dihapus dari sistem.
2. Sisi Sistem:
 - a. Validasi: Khusus untuk penambahan user baru, sistem akan melakukan Validasi Data untuk memastikan konsistensi dan keamanan input.
3. Sisi Database:
 - a. Persistensi Data: Database menjalankan perintah sesuai aksi yang dipilih, baik itu Simpan, Update, maupun Hapus data user.
4. Finalisasi: Admin diarahkan kembali ke halaman dashboard setelah aksi selesai dilakukan.

2.3.6 Activity Diagram Lihat Log Sensor (Admin)

Activity Diagram ini menggambarkan prosedur teknis yang dilakukan oleh aktor Admin untuk memantau riwayat data yang masuk ke dalam sistem. Berbeda dengan fitur cuaca umum, proses ini melibatkan interaksi langsung dengan penyimpanan internal (database) guna menyajikan data historis sensor secara akurat.



Gambar 8 Activity Diagram Lihat Log Sensor (Admin)

title Activity Diagram - Lihat Log Sensor

|Admin|

start

:Login;

:Pilih menu log sensor;

|Sistem|

:Ambil data sensor;

|Database|

:Query log sensor;

|Sistem|

:Tampilkan riwayat data;

|Admin|

stop

@enduml

A. Deskripsi Alur Kerja

Proses ini melibatkan tiga entitas utama yang bekerja secara berurutan:

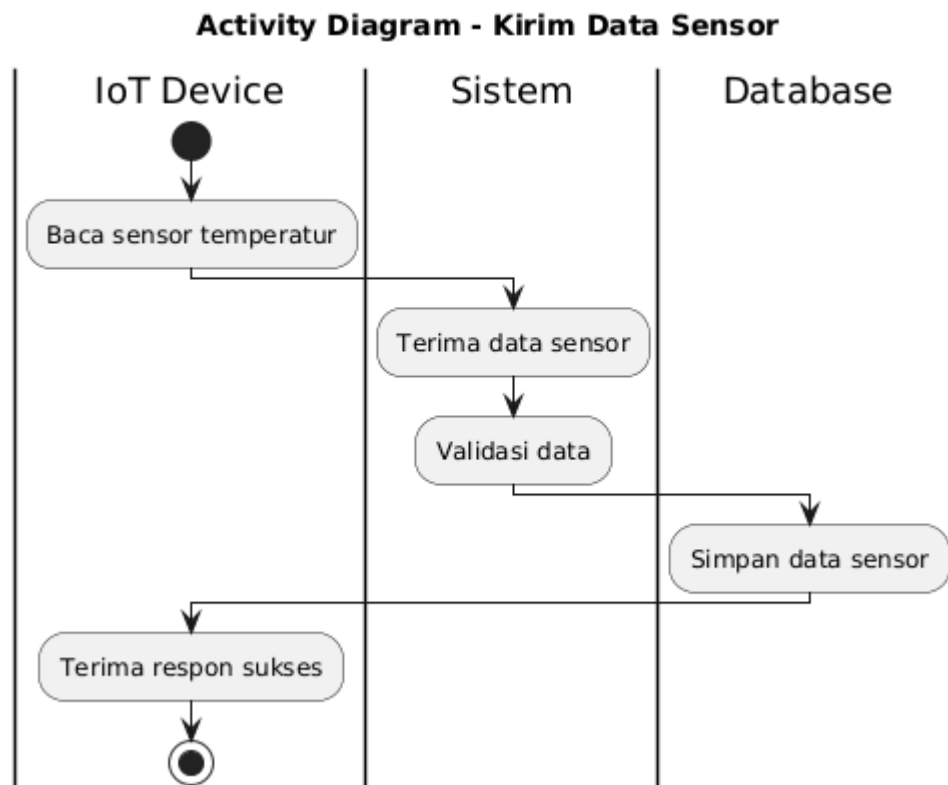
1. Sisi Admin:
 - a. Login: Admin memulai dengan melakukan autentikasi untuk mendapatkan hak akses ke data teknis.
 - b. Pilih Menu Log Sensor: Admin mengakses menu khusus yang disediakan untuk memantau catatan aktivitas sensor.
2. Sisi Sistem:
 - a. Ambil Data Sensor: Sistem menerima perintah dari Admin dan meneruskannya sebagai permintaan pengambilan data.
 - b. Tampilkan Riwayat Data: Setelah data diterima dari database, sistem menyajikannya dalam bentuk tabel atau daftar riwayat yang dapat dibaca oleh Admin.
3. Sisi Database:
 - a. Query Log Sensor: Database menjalankan perintah pencarian (*query*) untuk menarik seluruh catatan log sensor yang tersimpan dalam penyimpanan permanen.

B. Simbol Utama pada Diagram

1. Swimlane (Admin, Sistem, Database): Memperjelas pemisahan tanggung jawab antara tindakan pengguna, pemrosesan aplikasi, dan manajemen data pada level penyimpanan.
2. Initial Node: Titik awal ketika Admin mengakses dashboard untuk memantau log.
3. Final Node: Titik akhir proses setelah Admin selesai meninjau riwayat data.

2.3.7 Activity Diagram Kirim Data Sensor (IOT)

Activity Diagram ini menggambarkan proses otomatis pengumpulan data suhu yang dilakukan oleh perangkat keras (IoT device) ke dalam sistem. Berbeda dengan aktivitas sebelumnya, proses ini berjalan secara latar belakang tanpa melibatkan interaksi langsung dari pengguna.



Gambar 9 Activity Diagram Kirim Data Sensor (IOT)

@startuml

title Activity Diagram - Kirim Data Sensor

|IoT Device|

start

:Baca sensor temperatur;

|Sistem|

:Terima data sensor;

:Validasi data;

|Database|

:Simpan data sensor;

|IoT Device|

:Terima respon sukses;

stop

@enduml

A. Deskripsi Alur Kerja

Proses pengiriman data melibatkan tiga entitas utama:

1. IoT Device:
 - a. Baca Sensor Temperatur: Perangkat melakukan pembacaan suhu dari lingkungan secara berkala.
 - b. Terima Respon Sukses: Tahap akhir di mana perangkat menerima konfirmasi bahwa data telah berhasil diproses oleh server.
2. Sistem:
 - a. Terima Data Sensor: Sistem menerima paket data yang dikirimkan oleh perangkat IoT.
 - b. Validasi Data: Sistem melakukan pengecekan format dan integritas data untuk memastikan informasi yang masuk akurat dan tidak korup.
3. Database:
 - a. Simpan Data Sensor: Setelah divalidasi, data suhu disimpan ke dalam basis data agar nantinya dapat diakses oleh Admin melalui menu *Log Sensor*.

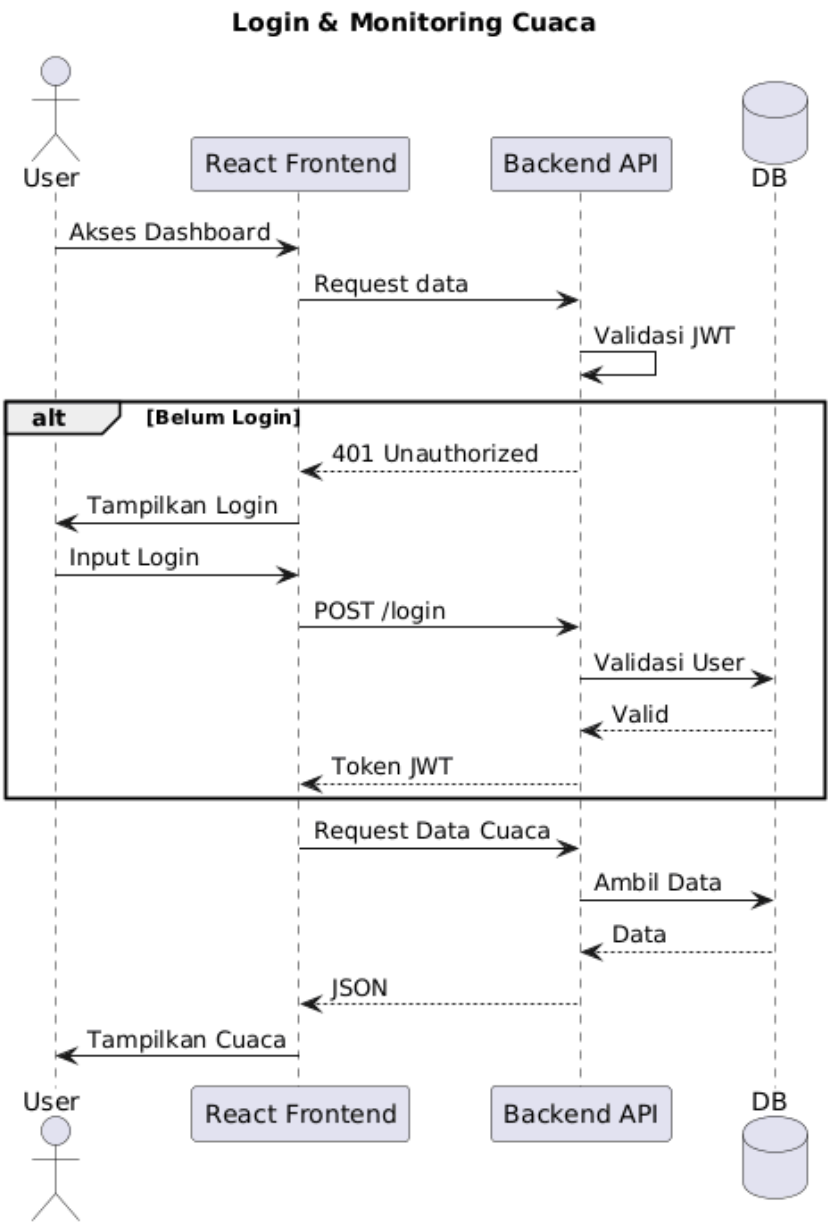
B. Simbol Utama pada Diagram

1. Swimlane (IoT Device, Sistem, Database): Menjelaskan distribusi tugas dalam siklus hidup data, mulai dari pengambilan (akuisisi), pemrosesan, hingga penyimpanan.
2. Initial Node: Titik awal pembacaan fisik oleh sensor.
3. Final Node: Akhir dari satu siklus pengiriman data.

2.4 Sequence Diagram

2.4.1 Sequence Diagram (User)

Sequence Diagram ini menggambarkan urutan pertukaran pesan dan data antar komponen sistem pada saat pengguna mengakses fitur monitoring cuaca. Diagram ini menekankan mekanisme keamanan menggunakan JWT (JSON Web Token) serta proses pengambilan data secara asynchronous.



Gambar 10 Sequence Diagram (User)

@startuml

title Sequence Diagram

title Login & Monitoring Cuaca

actor User

participant "React Frontend" as React

participant "Backend API" as API

database DB

User -> React : Akses Dashboard

React -> API : Request data

API -> API : Validasi JWT

alt Belum Login

API --> React : 401 Unauthorized

React -> User : Tampilkan Login

User -> React : Input Login

React -> API : POST /login

API -> DB : Validasi User

DB --> API : Valid

API --> React : Token JWT

end

React -> API : Request Data Cuaca

API -> DB : Ambil Data

DB --> API : Data

API --> React : JSON

React -> User : Tampilkan Cuaca

@enduml

A. Deskripsi Alur Interaksi

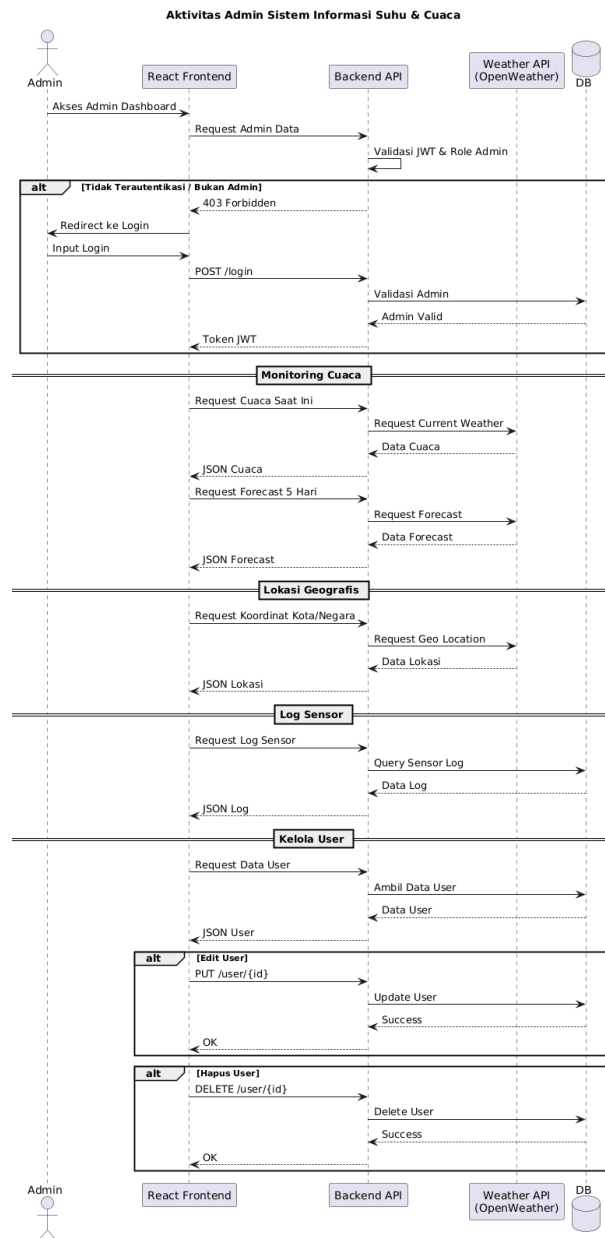
1. Inisiasi Akses:
 - a. User melakukan akses ke dashboard melalui React Frontend.
 - b. React Frontend mengirimkan permintaan data ke Backend API.
2. Validasi Keamanan (JWT):
 - a. Backend API melakukan validasi internal terhadap token JWT untuk memeriksa status autentikasi pengguna.
3. Kondisi Alternatif (Belum Login):
 - a. Jika pengguna belum terautentikasi, API mengirimkan respon 401 Unauthorized.
 - b. React Frontend secara otomatis mengarahkan User ke tampilan login.
 - c. User memasukkan kredensial yang kemudian dikirim via POST /login ke API.
 - d. Backend API melakukan verifikasi ke Database. Jika valid, API mengirimkan kembali Token JWT ke Frontend untuk disimpan sebagai sesi aktif.
4. Pengambilan Data Cuaca:
 - a. Setelah proses autentikasi berhasil (atau jika sudah login sebelumnya), Frontend melakukan Request Data Cuaca ke API.
 - b. API menarik informasi yang diperlukan dari Database.
 - c. Data dikembalikan dalam format JSON ke Frontend.
5. Output:
 - a. React Frontend mengolah data tersebut dan menampilkannya kepada User di layar dashboard.

B. Komponen Teknis dalam Diagram

1. User (Actor): Subjek yang berinteraksi dengan antarmuka pengguna.
2. React Frontend (Lifeline): Komponen antarmuka yang menangani logika presentasi dan permintaan klien.
3. Backend API (Lifeline): Komponen pelayan (*server*) yang menangani logika bisnis dan validasi keamanan.
4. DB (Database): Tempat penyimpanan data pengguna dan log informasi.

2.4.2 Sequence Diagram (Admin)

Sequence Diagram ini menggambarkan alur kerja teknis aktor Admin dalam mengelola seluruh fitur sistem. Diagram ini menekankan integrasi antara sistem internal dengan layanan pihak ketiga (OpenWeather API) serta penerapan kontrol akses yang ketat.



Gambar 11 Sequence Diagram (Admin)

@startuml

title Sequence Diagram

title Aktivitas Admin Sistem Informasi Suhu & Cuaca

actor Admin

participant "React Frontend" as React

participant "Backend API" as API

participant "Weather API\n(OpenWeather)" as Weather
database DB

Admin -> React : Akses Admin Dashboard

React -> API : Request Admin Data

API -> API : Validasi JWT & Role Admin

alt Tidak Terautentikasi / Bukan Admin

API --> React : 403 Forbidden

React -> Admin : Redirect ke Login

Admin -> React : Input Login

React -> API : POST /login

API -> DB : Validasi Admin

DB --> API : Admin Valid

API --> React : Token JWT

end

== Monitoring Cuaca ==

React -> API : Request Cuaca Saat Ini

API -> Weather : Request Current Weather

Weather --> API : Data Cuaca

API --> React : JSON Cuaca

React -> API : Request Forecast 5 Hari

API -> Weather : Request Forecast

Weather --> API : Data Forecast

API --> React : JSON Forecast

== Lokasi Geografis ==

React -> API : Request Koordinat Kota/Negara

API -> Weather : Request Geo Location

Weather --> API : Data Lokasi

API --> React : JSON Lokasi

== Log Sensor ==

React -> API : Request Log Sensor

API -> DB : Query Sensor Log

DB --> API : Data Log

API --> React : JSON Log

== Kelola User ==

React -> API : Request Data User

API -> DB : Ambil Data User

DB --> API : Data User

API --> React : JSON User

alt Edit User

React -> API : PUT /user/{id}

API -> DB : Update User

DB --> API : Success

API --> React : OK

end

alt Hapus User

React -> API : DELETE /user/{id}

API -> DB : Delete User

DB --> API : Success

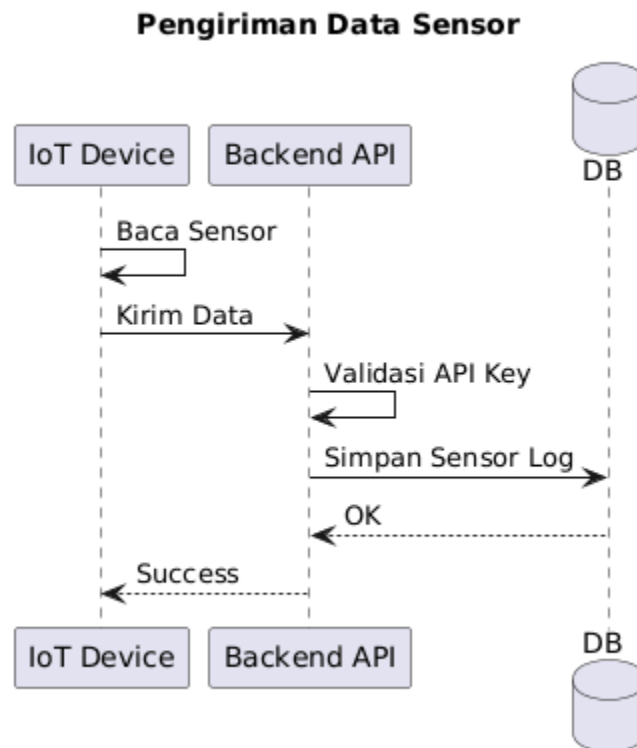
API --> React : OK

end

@enduml

2.4.3 Sequence Diagram (IOT)

Sequence Diagram ini menggambarkan alur pertukaran pesan dalam proses otomatisasi pengiriman data dari perangkat keras (IoT device) ke server. Berbeda dengan interaksi pengguna, proses ini mengandalkan mekanisme keamanan berbasis API Key untuk menjamin integritas data.



Gambar 12 Sequence Diagram Pengiriman Data Sensor (IOT)

@startuml

title Sequence Diagram

title Pengiriman Data Sensor

participant "IoT Device" as IoT

participant "Backend API" as API

database DB

IoT -> IoT : Baca Sensor

IoT -> API : Kirim Data

API -> API : Validasi API Key

API -> DB : Simpan Sensor Log

DB --> API : OK

API --> IoT : Success

@enduml

A. Deskripsi Alur Interaksi

1. Autentikasi dan Otorisasi Role:
 - a. Saat Admin mengakses dashboard, Backend API melakukan validasi JWT dan pengecekan Role Admin.
 - b. Jika tidak valid (bukan Admin), sistem mengirim respon 403 Forbidden dan mengarahkan kembali ke halaman login.
 - c. Jika validasi berhasil, sistem memberikan akses penuh ke berbagai modul.
2. Modul Monitoring Cuaca & Lokasi (Integrasi API Eksternal):
 - a. Untuk fitur Cuaca Saat Ini, Forecast 5 Hari, dan Lokasi Geografis, Backend API bertindak sebagai jembatan yang mengirimkan permintaan ke Weather API (OpenWeather).
 - b. Data yang diterima dari OpenWeather kemudian dikonversi menjadi format JSON untuk ditampilkan pada React Frontend.
3. Modul Log Sensor (Manajemen Data Internal):
 - a. Sistem melakukan Query Sensor Log langsung ke Database internal untuk menarik riwayat data yang dikirim oleh perangkat IoT.
 - b. Data ini kemudian disajikan secara historis di dashboard Admin.
4. Modul Kelola User (Operasi CRUD):
 - a. Admin dapat melakukan manajemen pengguna, di mana sistem melakukan permintaan PUT untuk melakukan pembaruan (*Update*) data dan permintaan DELETE untuk menghapus user.
 - b. Setiap aksi dikonfirmasi oleh Database dengan respon *Success* sebelum Frontend menampilkan status berhasil.

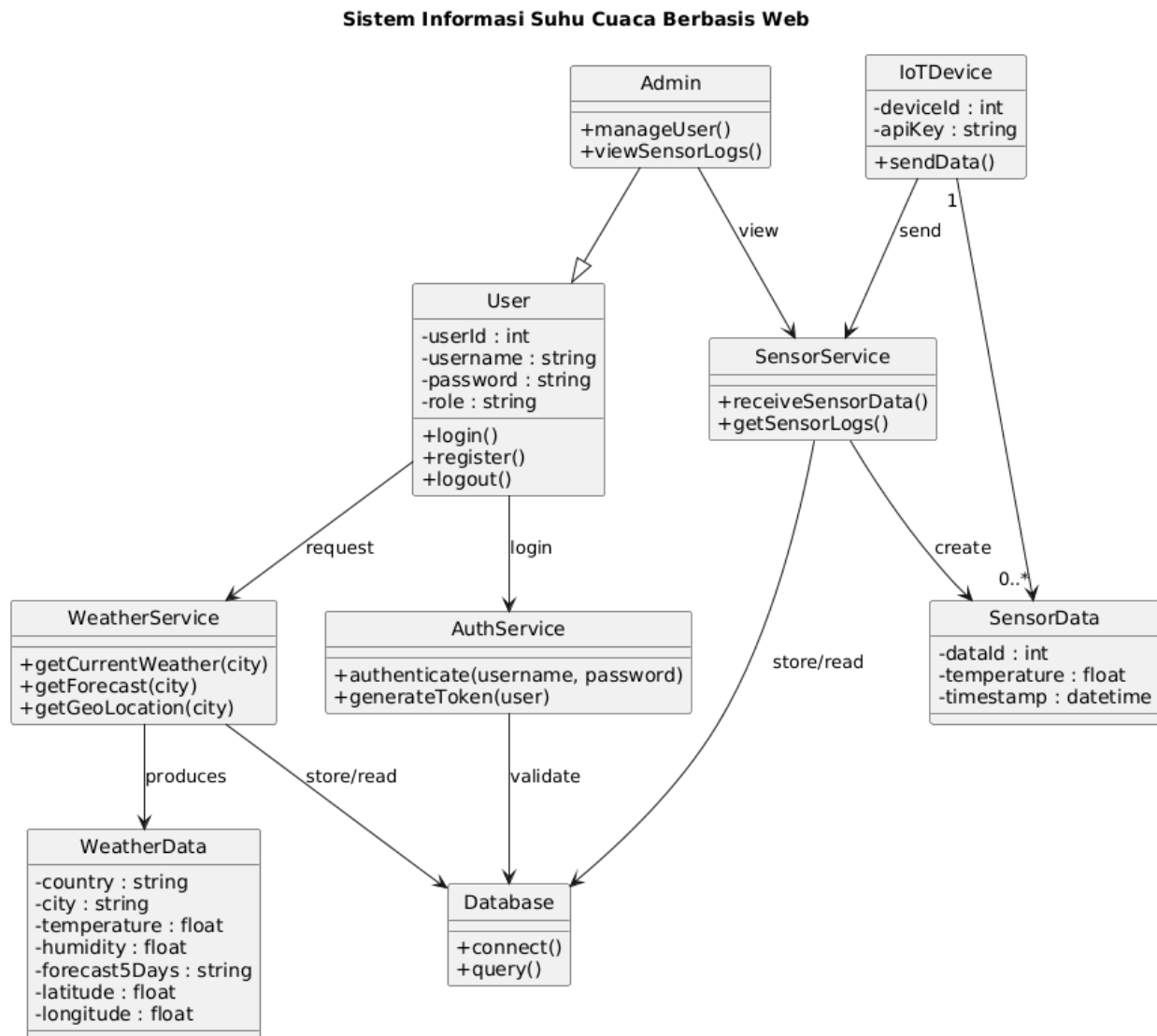
B. Komponen Arsitektur dalam Diagram

1. Admin (Actor): Pengelola sistem dengan hak akses penuh.
2. React Frontend (Lifeline): Antarmuka pengguna berbasis web untuk interaksi visual.

3. Backend API (Lifeline): Pusat kendali logika bisnis dan keamanan.
4. Weather API (Lifeline): Penyedia data cuaca dan geolokasi eksternal.
5. DB (Database): Penyimpanan data user dan log sensor internal.

2.5 Class Diagram

Class Diagram ini menggambarkan struktur statis sistem dengan merinci kelas-kelas, atribut, metode, serta relasi antar kelas. Arsitektur sistem menerapkan pendekatan Service-Oriented, di mana logika bisnis dipisahkan ke dalam layanan-layanan khusus.



Gambar 14 Class Diagram Sistem Informasi Suhu dan Cuaca

title Class Diagram

title Sistem Informasi Suhu Cuaca Berbasis Web

skinparam classAttributeIconSize 0

skinparam style strictuml

' =====

' USER & ROLE

' =====

class User {

- userId : int

- username : string

- password : string

- role : string

+ login()

+ register()

+ logout()

}

class Admin {

+ manageUser()

+ viewSensorLogs()

}

Admin --|> User

' =====

' AUTHENTICATION

' =====

```
class AuthService {  
    + authenticate(username, password)  
    + generateToken(user)  
}
```

User --> AuthService : login

```
' =====  
  
' WEATHER (CUACA GLOBAL)  
  
' =====
```

```
class WeatherService {  
    + getCurrentWeather(city)  
    + getForecast(city)  
    + getGeoLocation(city)  
}
```

```
class WeatherData {  
    - country : string  
    - city : string  
    - temperature : float  
    - humidity : float  
    - forecast5Days : string  
    - latitude : float  
    - longitude : float  
}
```

User --> WeatherService : request

WeatherService --> WeatherData : produces

' =====

' SENSOR & IOT

' =====

class IoTDevice {

- deviceId : int

- apiKey : string

+ sendData()

}

class SensorService {

+ receiveSensorData()

+ getSensorLogs()

}

class SensorData {

- dataId : int

- temperature : float

- timestamp : datetime

}

IoTDevice --> SensorService : send

SensorService --> SensorData : create

IoTDevice "1" --> "0..*" SensorData

Admin --> SensorService : view

' =====

' DATABASE

' =====

```

class Database {
    + connect()
    + query()
}

```

SensorService --> Database : store/read

WeatherService --> Database : store/read

AuthService --> Database : validate

@enduml

A. Deskripsi Komponen Kelas

1. Modul Pengguna & Autentikasi:
 - a. User: Kelas dasar yang menyimpan informasi profil dan metode dasar seperti login(), register(), dan logout().
 - b. Admin: Merupakan turunan dari kelas User (Inheritance), yang memiliki metode tambahan untuk fungsi manajerial seperti manageUser() dan viewSensorLogs().
 - c. AuthService: Layanan yang bertanggung jawab atas proses keamanan, termasuk autentikasi kredensial dan pembuatan token akses.
2. Modul Layanan Cuaca (External API):
 - a. WeatherService: Menangani permintaan data cuaca terkini, prakiraan 5 hari, dan geolokasi.
 - b. WeatherData: Objek data yang menyimpan hasil pemrosesan dari layanan cuaca, termasuk temperatur, kelembaban, dan koordinat.
3. Modul IoT & Sensor:
 - a. IoTDevice: Mewakili perangkat keras yang mengirimkan data ke sistem melalui metode sendData().
 - b. SensorService: Menerima data dari perangkat IoT dan mengelola log data sensor.
 - c. SensorData: Kelas data yang menyimpan informasi temperatur dan stempel waktu (*timestamp*) dari sensor.
4. Infrastruktur:
 - a. Database: Kelas pusat yang menangani konektivitas dan eksekusi kueri untuk semua layanan.

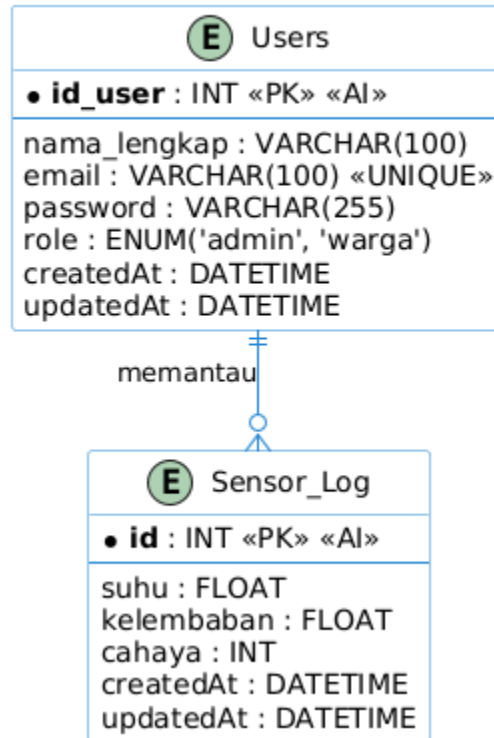
B. Hubungan Antar Kelas (Relationships)

1. Inheritance (Pewarisan): Admin mewarisi seluruh sifat dari User.
2. Dependency/Association: User menggunakan WeatherService untuk meminta data, sementara IoTDevice mengirimkan data ke SensorService.
3. Multiplicity: Satu IoTDevice dapat menghasilkan banyak (0..*) SensorData.

2.6 Entity Relationship Diagram (ERD)

ERD ini menggambarkan arsitektur data pada Sistem Monitoring IoT yang berfokus pada manajemen pengguna serta penyimpanan data log sensor. Struktur data ini dirancang untuk mendukung efisiensi kueri dan menjaga integritas data.

ERD Sistem Monitoring IoT (Users & Sensor Logs)



Gambar 15 ERD Sistem Monitoring IoT (User dan Sensor Logs)

@startuml

title ERD Sistem Monitoring IoT (Users & Sensor Logs)

' Pengaturan tampilan agar lebih profesional

skinparam linetype ortho

skinparam shadowing false

skinparam class {

 BackgroundColor White

 ArrowColor #2688d4

 BorderColor #2688d4

}

entity "Users" as User {

 * **id_user** : INT <<PK>> <<AI>>

 --

 nama_lengkap : VARCHAR(100)

 email : VARCHAR(100) <<UNIQUE>>

 password : VARCHAR(255)

 role : ENUM('admin', 'warga')

 createdAt : DATETIME

 updatedAt : DATETIME

}

entity "Sensor_Log" as Sensor {

 * **id** : INT <<PK>> <<AI>>

 --

 suhu : FLOAT

 kelembaban : FLOAT

 cahaya : INT

 createdAt : DATETIME

 updatedAt : DATETIME

}

' Relasi: User memantau log sensor

User ||--o{ Sensor : "memantau"

@enduml

A. Deskripsi Entitas dan Atribut

Sistem ini memiliki dua entitas utama:

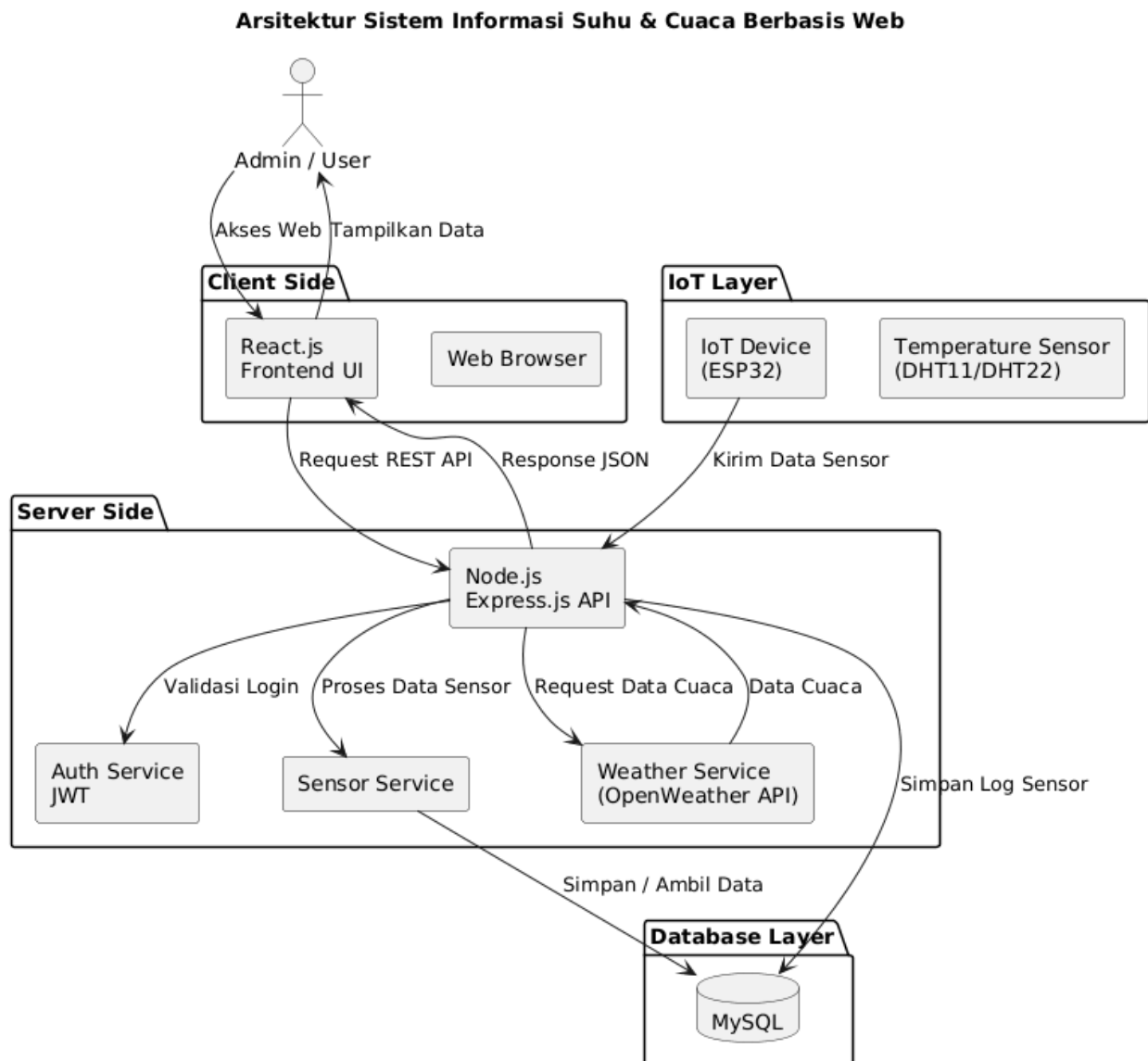
1. Entitas Users (Pengguna):
 - a. id_user (Primary Key): Identitas unik otomatis untuk setiap pengguna.
 - b. nama_lengkap: Nama pengguna yang terdaftar.
 - c. email (Unique): Alamat surel sebagai identitas login unik.
 - d. password: Kredensial keamanan yang tersimpan dalam format terenkripsi.
 - e. role (Enum): Klasifikasi hak akses, yaitu admin (memiliki kontrol penuh) atau warga (pengguna umum).
 - f. createdAt & updatedAt: Penanda waktu untuk audit pembuatan dan pembaruan akun.
2. Entitas Sensor_Log (Catatan Sensor):
 - a. id (Primary Key): Identitas unik untuk setiap catatan data sensor yang masuk.
 - b. suhu (Float): Data numerik hasil pembacaan temperatur.
 - c. kelembaban (Float): Data numerik hasil pembacaan kadar air di udara.
 - d. cahaya (Int): Data numerik hasil pembacaan intensitas cahaya.
 - e. createdAt & updatedAt: Penanda waktu presisi kapan data sensor diterima dan disimpan oleh sistem.

B. Relasi Antar Entitas

- Relationship: Users "memantau" Sensor_Log.
- Kardinalitas (One-to-Many): Satu pengguna (User) dapat memantau banyak catatan sensor (Sensor_Log), namun setiap catatan sensor secara logis merupakan bagian dari kumpulan data yang dipantau oleh pengguna sistem. Hal ini disimbolkan dengan notasi *crows foot* ||--o{ pada diagram.

2.7 Arsitektur Diagram

Arsitektur sistem ini menerapkan pola N-Tier Architecture dengan pemisahan tanggung jawab antara lapisan antarmuka pengguna, logika server, perangkat IoT, dan penyimpanan data. Pendekatan ini memungkinkan sistem bersifat modular, lebih aman, serta mudah dikembangkan dan dipelihara di masa mendatang.



Gambar 16 Arsitektur Diagram Sistem Informasi Suhu dan Cuaca

@startuml

title Arsitektur Sistem Informasi Suhu & Cuaca Berbasis Web

skinparam componentStyle rectangle

skinparam shadowing false

actor "Admin / User" as User

package "Client Side" {

```

    component "Web Browser"

    component "React.js\nFrontend UI" as React
}

package "Server Side" {

    component "Node.js\nExpress.js API" as API

    component "Auth Service\nJWT" as Auth

    component "Weather Service\n(OpenWeather API)" as Weather

    component "Sensor Service" as SensorService

}

package "IoT Layer" {

    component "Temperature Sensor\n(DHT11/DHT22)" as Sensor

    component "IoT Device\n(ESP32)" as IoT

}

package "Database Layer" {

    database "MySQL" as DB

}

```

User --> React : Akses Web

React --> API : Request REST API

API --> Auth : Validasi Login

API --> Weather : Request Data Cuaca

API --> SensorService : Proses Data Sensor

SensorService --> DB : Simpan / Ambil Data

Weather --> API : Data Cuaca

API --> React : Response JSON

React --> User : Tampilkan Data

IoT --> API : Kirim Data Sensor

API --> DB : Simpan Log Sensor

@enduml

A. Lapisan Arsitektur (Layers)

Berdasarkan diagram, sistem dibagi menjadi empat lapisan utama:

1. Client Side (Sisi Pengguna):
 - a. Web Browser: Media utama bagi pengguna untuk berinteraksi dengan sistem.
 - b. React.js Frontend UI: Kerangka kerja yang menangani tampilan antarmuka (User Interface) dan mengelola permintaan ke server secara dinamis.
2. Server Side (Logika Bisnis):
 - a. Node.js & Express.js API: Berperan sebagai pusat kendali (backend) yang memproses permintaan REST API dari klien.
 - b. Auth Service (JWT): Mengelola keamanan dan autentikasi pengguna menggunakan *JSON Web Token*.
 - c. Weather Service: Bertugas melakukan integrasi dengan pihak ketiga (OpenWeather API) untuk mendapatkan data cuaca.
 - d. Sensor Service: Mengelola logika penerimaan dan pengolahan data yang masuk dari perangkat IoT.
3. IoT Layer (Perangkat Keras):
 - a. ESP32: Perangkat mikrokontroler yang berfungsi sebagai jembatan komunikasi antara sensor dan server.
 - b. DHT11/DHT22: Sensor fisik yang membaca parameter suhu dan kelembapan di lokasi.
4. Database Layer (Penyimpanan):
 - a. MySQL: Sistem manajemen basis data relasional yang menyimpan informasi akun pengguna dan catatan historis sensor (*sensor logs*).

B. Alur Kerja Komunikasi Data

1. Interaksi Manusia: Pengguna mengakses UI (React), yang kemudian mengirimkan permintaan ke API. API akan memvalidasi identitas melalui Auth Service sebelum menarik data dari Database atau OpenWeather API. Hasilnya dikirimkan kembali ke pengguna dalam format JSON.

2. Interaksi Mesin (IoT): Perangkat ESP32 mengirimkan data sensor langsung ke API secara berkala. API kemudian memvalidasi data tersebut dan menyimpannya secara otomatis ke dalam Database MySQL.

BAB III – APLIKASI

3.1 Deskripsi Alat dan Implementasi Perangkat Keras

Perangkat keras digunakan sebagai komponen utama dalam proses pengambilan data suhu dan kelembapan udara. Sistem IoT dirancang agar mampu bekerja secara otomatis, membaca kondisi lingkungan secara periodik, mengolah data awal, kemudian mengirimkannya ke server backend untuk disimpan dan ditampilkan pada aplikasi web.

3.1.1 Perangkat Keras yang Digunakan

1. **ESP32**

ESP32 berfungsi sebagai mikrokontroler utama sekaligus pusat kendali sistem IoT. ESP32 bertugas melakukan inisialisasi sistem dan sensor, membaca data suhu dan kelembapan, mengolah data mentah menjadi data digital, menghubungkan sistem ke jaringan WiFi, serta mengirimkan data ke server backend menggunakan HTTP request dalam format JSON. Selain itu, ESP32 juga menangani proses reconnect otomatis ketika koneksi internet terputus agar sistem tetap berjalan stabil dan berkelanjutan.



Gambar 17 alat ESP32

2. **Sensor Temperatur dan Humidity (DHT11/DHT22)**

Sensor DHT berfungsi sebagai alat pembaca kondisi lingkungan. Sensor ini mengukur suhu udara dalam satuan derajat Celcius dan kelembapan udara dalam satuan persentase. Data hasil pembacaan sensor dikirimkan ke ESP32 untuk diproses dan diteruskan ke server. Sensor ini menjadi sumber utama data cuaca dalam sistem sehingga akurasi sistem sangat bergantung pada kinerja sensor.



Gambar 18 Sensor Temperatur dan Humidity

3. **Kabel**

jumper

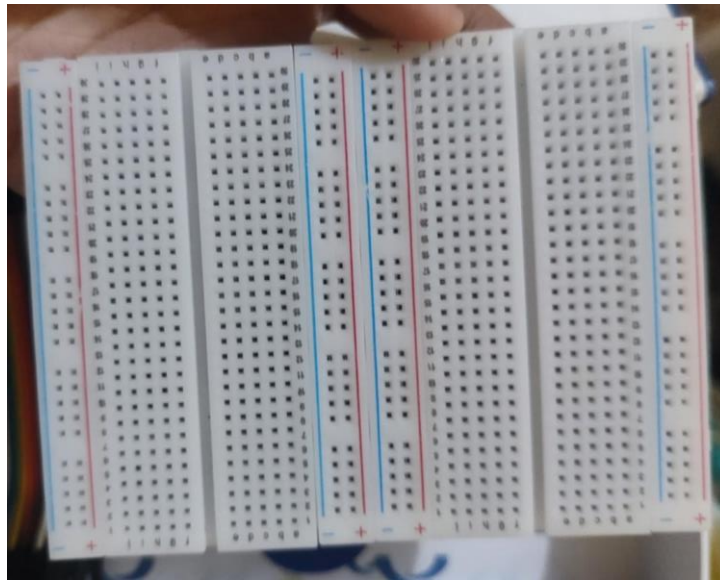
Kabel jumper digunakan sebagai media penghubung antara pin sensor dan pin ESP32. Kabel ini memastikan aliran daya dan data berjalan dengan stabil dari sensor menuju mikrokontroler serta memudahkan proses perakitan dan perawatan rangkaian.



Gambar 19 Kabel Jumper

4. **Breadboard**

Breadboard digunakan sebagai media perakitan rangkaian tanpa penyolderan. Penggunaan breadboard mempermudah proses pengujian, perbaikan, dan pengembangan sistem karena komponen dapat dilepas dan dipasang kembali tanpa merusak rangkaian.



Gambar 20 Breadboard

5. **Power supply / kabel USB**

Power supply atau kabel USB digunakan untuk menyuplai daya ke ESP32. Sumber daya

yang stabil sangat penting agar sistem dapat berjalan secara kontinu dan pembacaan sensor tidak terganggu.

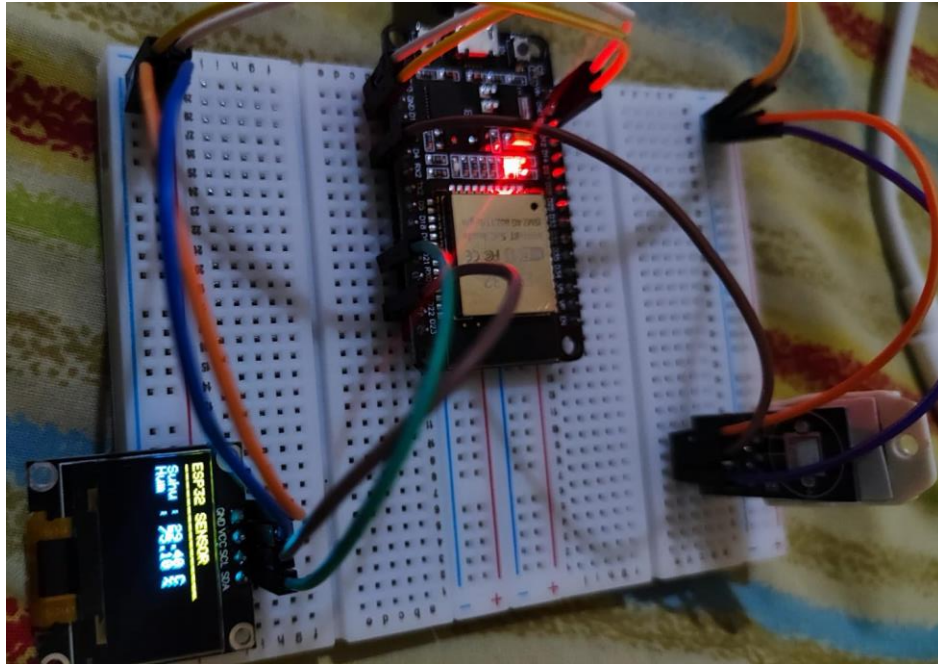


Gambar 21 Kabel USB

3.1.2 Alur Kerja Perangkat Keras

1. ESP32 melakukan proses inialisasi sistem, termasuk konfigurasi pin, inialisasi sensor, dan koneksi ke jaringan WiFi.
2. Sensor membaca suhu dan kelembapan udara dari lingkungan sekitar sesuai interval waktu yang ditentukan.
3. ESP32 menerima dan memproses data hasil pembacaan sensor.
4. ESP32 mengirimkan data suhu dan kelembapan ke backend server menggunakan HTTP request.
5. Backend menerima data, melakukan validasi, lalu menyimpannya ke dalam database.
6. Proses pembacaan dan pengiriman data diulang secara periodik sehingga sistem mampu melakukan monitoring secara real-time dan berkelanjutan.

3.1.3 Foto Alat



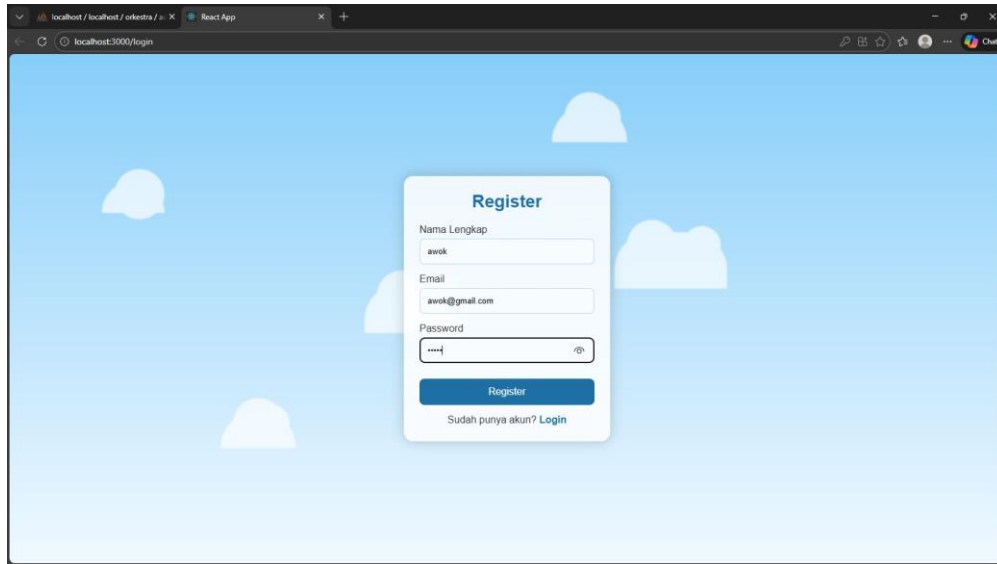
Gambar 22 Gambar Alat

1. Foto rangkaian alat menunjukkan hubungan antara ESP32, sensor DHT, kabel jumper, dan breadboard.
2. Rangkaian ini digunakan sebagai alat pembaca suhu dan kelembapan yang terhubung ke sistem web.
3. Foto alat menjadi bukti implementasi nyata sistem IoT sebagai pengumpul data lingkungan.

3.2 Tampilan Aplikasi Web

Aplikasi web digunakan sebagai media utama untuk menampilkan data sensor, informasi cuaca, serta pengelolaan sistem. Aplikasi ini memungkinkan pengguna mengakses informasi cuaca melalui browser, sedangkan admin dapat memantau log sensor dan mengelola data pengguna.

3.2.1 Register User



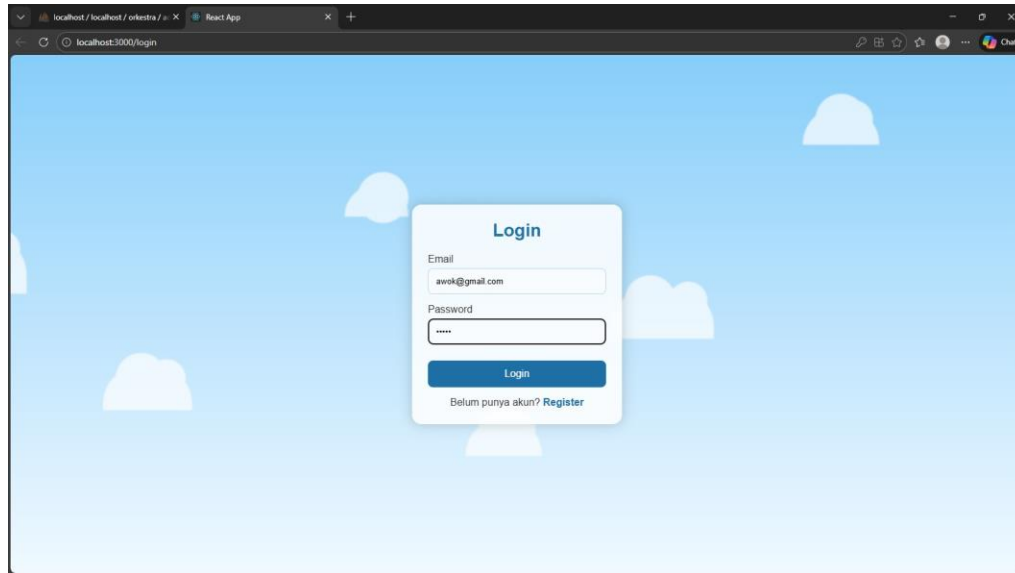
Gambar 23 Tampilan Registrasi User

Halaman register user digunakan untuk mendaftarkan pengguna baru ke dalam sistem. Fitur dan alur kerja pada halaman register user adalah sebagai berikut:

- Pengguna mengisi form pendaftaran yang terdiri dari nama lengkap, email, dan password.
- Sistem melakukan validasi data untuk memastikan seluruh field terisi dengan benar.
- Setelah data valid, sistem mengirimkan data ke backend untuk disimpan ke dalam database.
- Jika registrasi berhasil, pengguna diarahkan ke halaman login untuk masuk ke sistem.
- Jika terjadi kesalahan (misalnya email sudah terdaftar), sistem akan menampilkan pesan error.

Halaman ini berfungsi sebagai pintu awal agar masyarakat dapat memiliki akun dan mengakses layanan sistem.

3.2.2 Login User



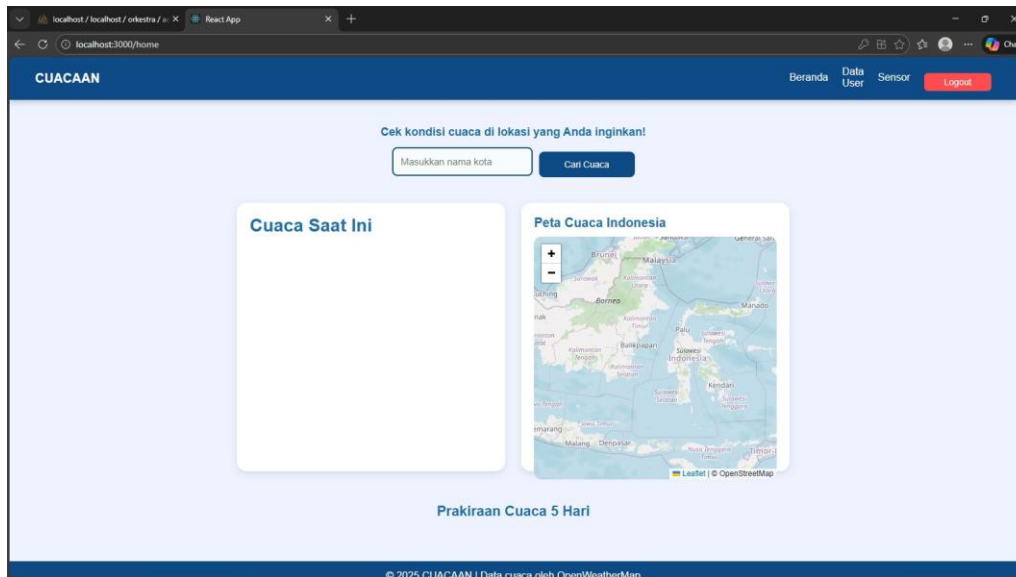
Gambar 24 Tampilan Login User

Halaman login user digunakan untuk proses autentikasi pengguna sebelum mengakses sistem. Fitur dan alur kerja pada halaman login user adalah sebagai berikut:

- Pengguna memasukkan email dan password yang telah didaftarkan.
- Sistem memverifikasi data login ke backend.
- Jika data sesuai, sistem akan membuat sesi login dan mengarahkan pengguna ke halaman beranda (homepage).
- Jika data tidak sesuai, sistem akan menampilkan notifikasi bahwa email atau password salah.

Halaman login berfungsi sebagai mekanisme keamanan agar hanya pengguna terdaftar yang dapat mengakses sistem.

3.2.3 Homepage User



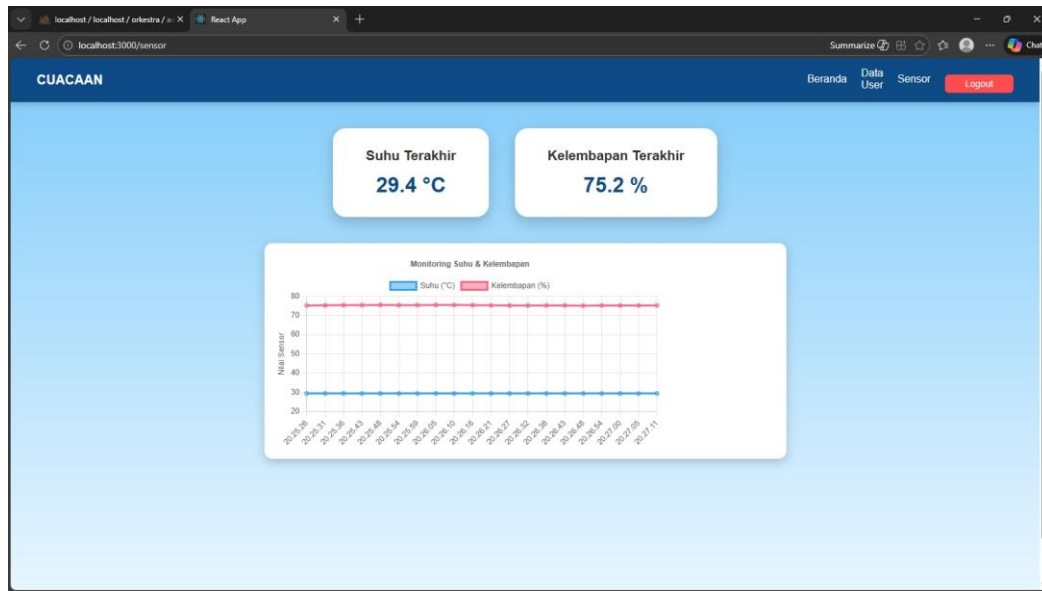
Gambar 25 Tampilan Homepage User

Halaman homepage merupakan halaman utama setelah user berhasil login. Fungsi utama homepage adalah:

- Menampilkan judul dan identitas sistem CUACAAN.
- Menyediakan fitur pencarian cuaca berdasarkan nama kota.
- Menampilkan informasi cuaca saat ini.
- Menampilkan peta cuaca Indonesia untuk memberikan gambaran kondisi cuaca secara geografis.
- Menjadi pusat navigasi menuju halaman data sensor dan halaman lain melalui menu navbar.

Homepage berfungsi sebagai dashboard awal untuk memberikan ringkasan informasi cuaca kepada pengguna.

3.2.4 Data Sensor User



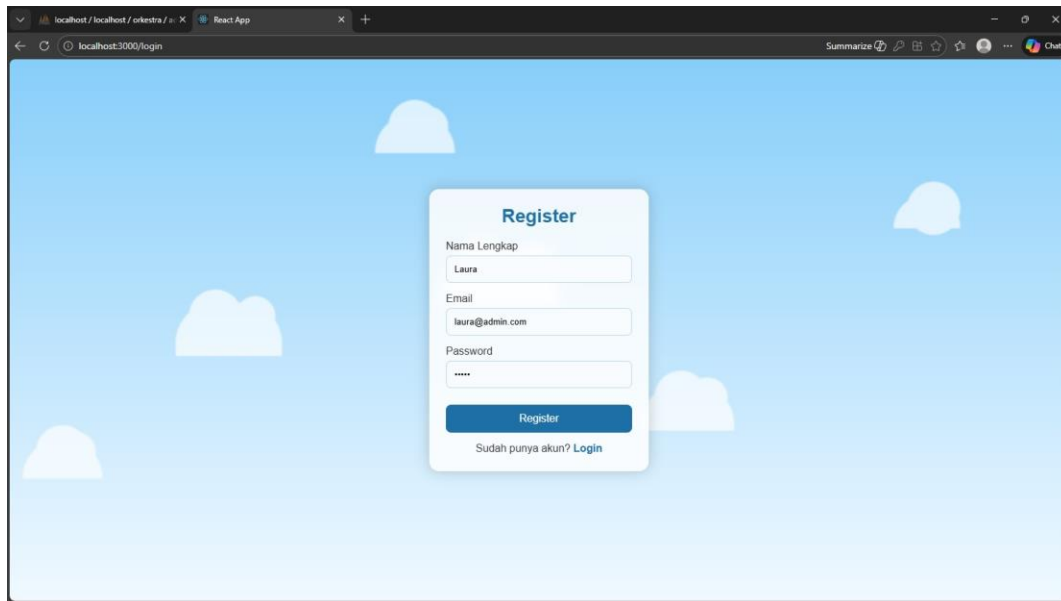
Gambar 26 Tampilan Data Sensor User

Halaman data sensor digunakan untuk menampilkan hasil pembacaan sensor IoT. Fitur dan informasi yang ditampilkan pada halaman ini meliputi:

- Menampilkan nilai suhu terakhir (°C) yang dikirim oleh sensor.
- Menampilkan nilai kelembapan terakhir (%) yang dikirim oleh sensor.
- Menyediakan grafik monitoring suhu dan kelembapan berdasarkan waktu untuk melihat perubahan data secara periodik.
- Data diperoleh secara real-time dari backend yang terhubung langsung dengan ESP32.

Halaman ini memungkinkan pengguna memantau kondisi lingkungan secara visual dan mudah dipahami.

3.2.5 Register Admin



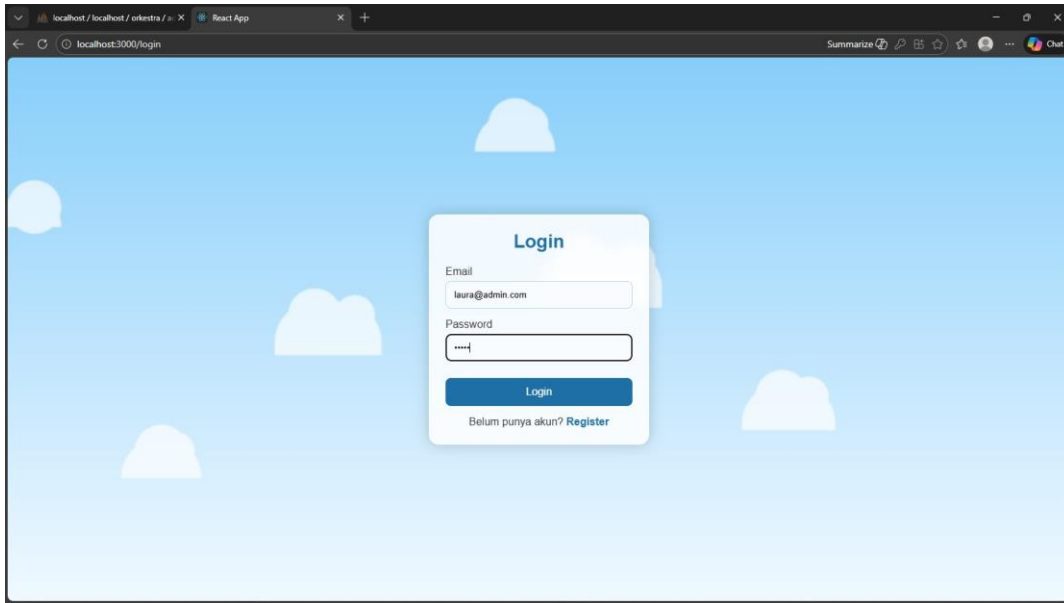
Gambar 27 Tampilan Register Admin

Halaman register admin digunakan untuk membuat akun dengan hak akses admin. Fungsi halaman ini meliputi:

- Mendaftarkan akun admin dengan nama, email, dan password.
- Data admin disimpan di database dengan role sebagai admin.
- Akun admin memiliki hak akses khusus untuk mengelola data pengguna.

Halaman ini tidak ditujukan untuk pengguna umum, melainkan untuk pengelola sistem.

3.2.6 Login Admin



Gambar 28 Tampilan Login Admin

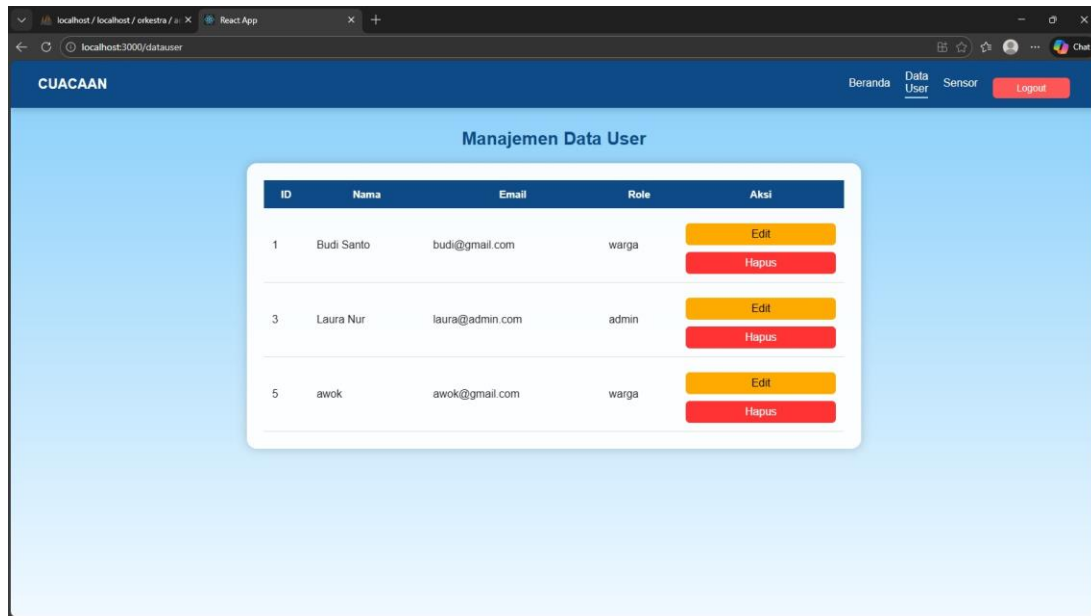
Halaman login admin digunakan untuk autentikasi admin sebelum masuk ke sistem manajemen.

Fungsi halaman ini meliputi:

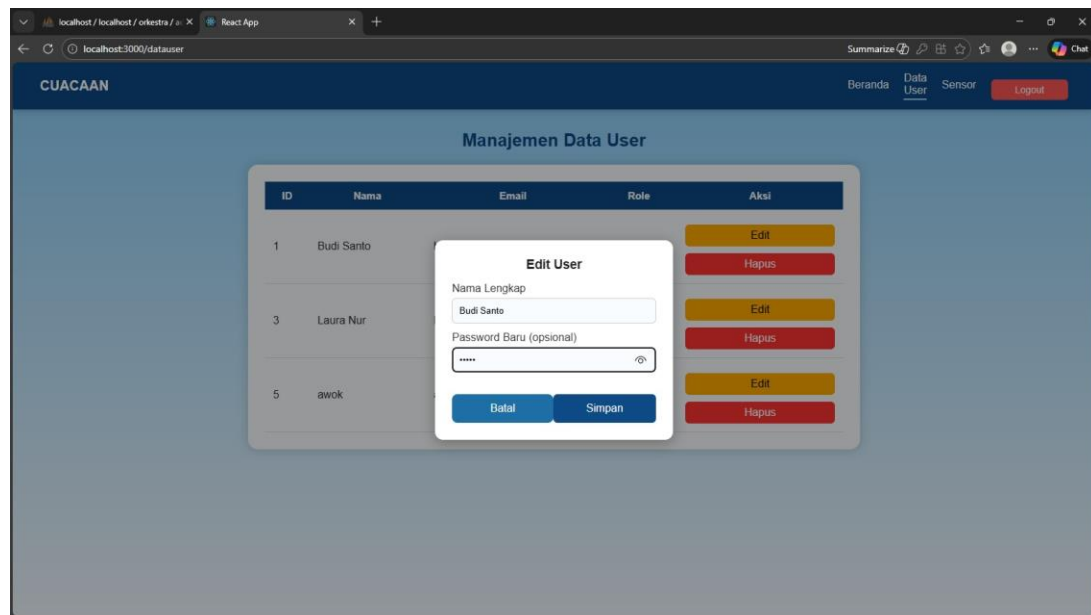
- Admin memasukkan email dan password.
- Sistem memverifikasi akun dan role pengguna.
- Jika role adalah admin, maka admin diarahkan ke halaman manajemen data user.
- Jika bukan admin, sistem menolak akses ke halaman pengelolaan data.

Halaman ini berfungsi sebagai lapisan keamanan agar hanya admin yang dapat mengakses fitur manajemen sistem.

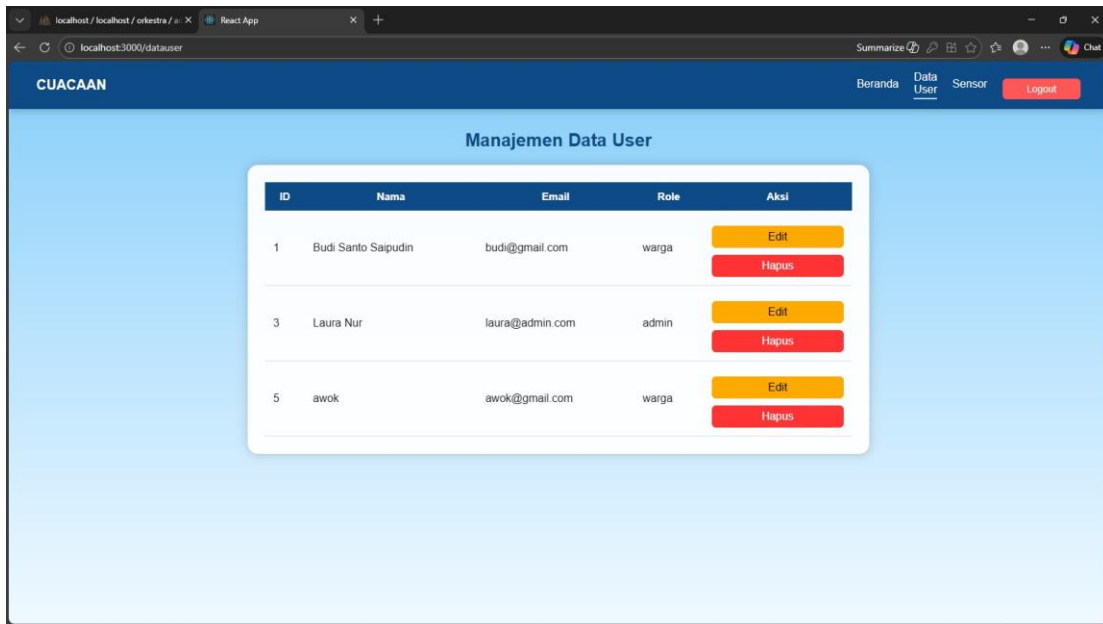
3.2.7 Manage Data User untuk Admin



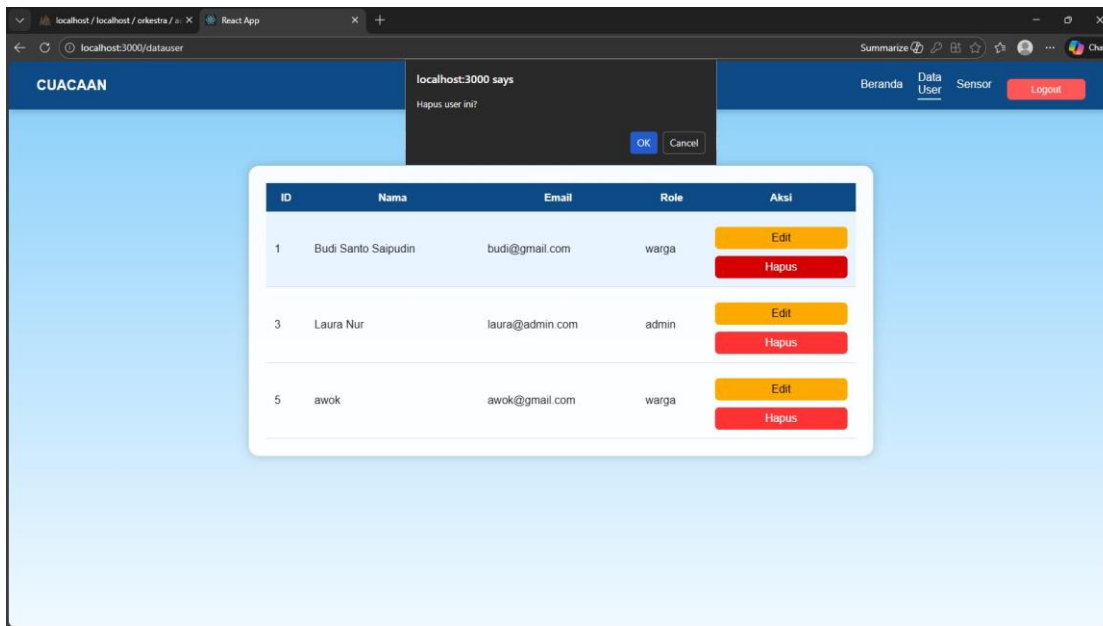
Gambar 29 Tampilan Halaman Manajemen Data User



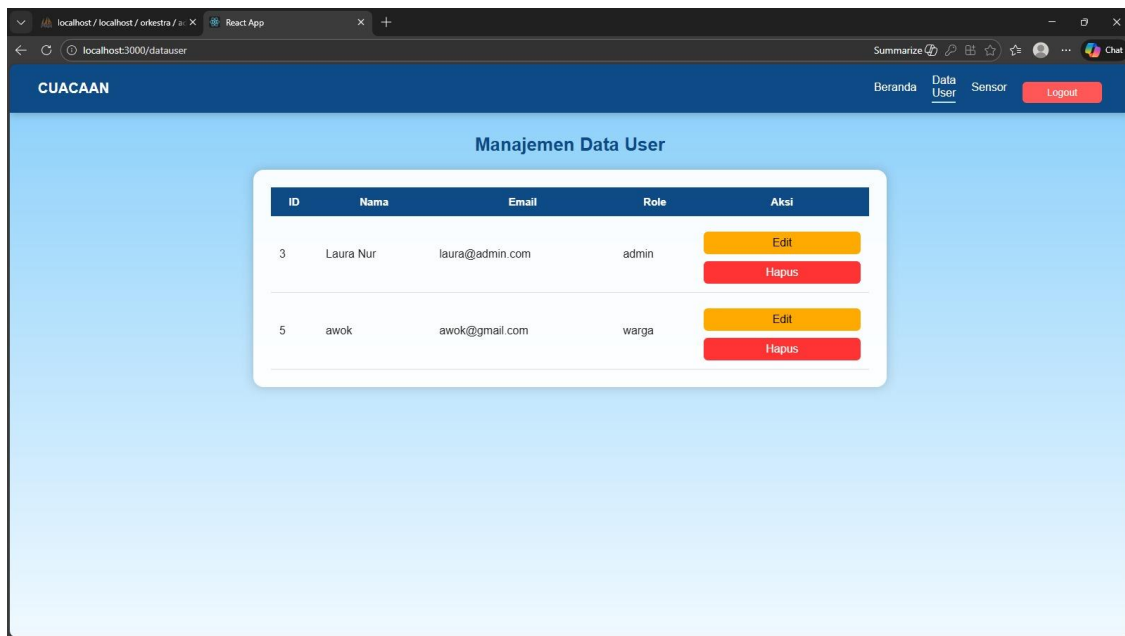
Gambar 30 Tampilan Halaman Edit User



Gambar 31 Tampilan Setelah Edit



Gambar 32 Tampilan Hapus User



Gambar 33 Tampilan Setelah Hapus

Halaman manajemen data user digunakan oleh admin untuk mengelola seluruh data pengguna. Fitur pada halaman ini meliputi:

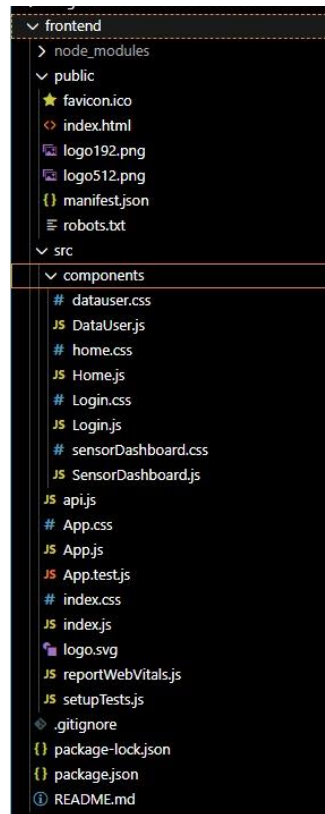
- Menampilkan daftar seluruh user dalam bentuk tabel (ID, nama, email, dan role).
- Admin dapat melakukan edit data user, seperti mengubah nama dan password.
- Admin dapat melakukan hapus user dengan konfirmasi terlebih dahulu untuk mencegah kesalahan penghapusan.
- Perubahan data akan langsung tersimpan di database dan diperbarui di tampilan aplikasi.

Halaman ini berfungsi sebagai pusat kontrol admin untuk menjaga keteraturan dan validitas data pengguna dalam sistem.

3.3 Struktur Folder dan Source Code Frontend

Frontend dikembangkan menggunakan React.js dan berfungsi sebagai antarmuka pengguna. Frontend menangani tampilan visual, interaksi pengguna, pengelolaan halaman, serta komunikasi dengan backend melalui REST API.

3.3.1 File dan Folder Root Frontend



Gambar 34 Tampilan Folder Frontend

1. `.gitignore`
Digunakan untuk mengecualikan file dan folder tertentu seperti `node_modules`, file konfigurasi lokal, dan file hasil build agar tidak masuk ke repository.
2. `package.json`
Berisi konfigurasi utama frontend, daftar library yang digunakan, serta script perintah untuk menjalankan, membangun, dan menguji aplikasi.
3. `package-lock.json`
Menyimpan detail versi library yang terinstal agar proses instalasi dependency tetap konsisten.
4. `README.md`
Berisi dokumentasi cara instalasi, struktur proyek, dan cara menjalankan aplikasi frontend.
5. `node_modules`
Berisi seluruh library frontend yang terinstal dan digunakan oleh aplikasi React.

3.3.2 Folder public

1. index.html
Merupakan template HTML utama tempat seluruh aplikasi React dirender.
2. favicon.ico
Digunakan sebagai ikon website pada tab browser.
3. manifest.json
Berisi konfigurasi aplikasi web seperti nama aplikasi, ikon, dan tema.
4. robots.txt
Digunakan untuk mengatur akses mesin pencari terhadap website.

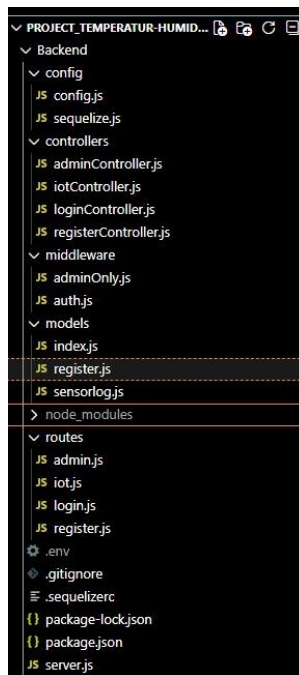
3.3.3 Folder src

1. index.js
Entry point React yang merender App.js ke halaman web.
2. App.js
Komponen utama yang mengatur struktur aplikasi dan pemanggilan seluruh komponen.
3. api.js
Mengatur seluruh komunikasi dengan backend, termasuk request login, pengambilan data sensor, pengambilan log, dan manajemen user.
4. App.css
Mengatur tampilan global aplikasi seperti layout dan warna.
5. index.css
Mengatur styling dasar seluruh halaman.
6. App.test.js
Digunakan untuk pengujian unit komponen aplikasi.
7. reportWebVitals.js
Digunakan untuk memantau performa aplikasi web.
8. setupTests.js
Digunakan untuk konfigurasi awal pengujian.
9. logo.svg
Digunakan sebagai aset grafis aplikasi.

3.3.4 Folder src/components

1. login.js
Mengatur halaman login, pengiriman data login ke backend, penerimaan token, dan pengalihan halaman.
2. login.css
Mengatur tampilan halaman login.

3. `sensordashboard.js`
Menampilkan data suhu, kelembapan, dan log sensor serta mengatur pembaruan data.
4. `sensordashboard.css`
Mengatur tampilan dashboard sensor.
5. `DataUser.js`
Menampilkan daftar user serta menyediakan fitur edit dan hapus user.
6. `datauser.css`
Mengatur tampilan halaman manajemen user.
7. `home.css`
Mengatur tampilan homepage.



Gambar 35 Tampilan Folder Backend

3.4 Struktur Folder dan Source Code Backend

Backend dikembangkan menggunakan Node.js dan Express.js. Backend berfungsi sebagai pusat logika sistem, pengelola autentikasi, penghubung database, serta penerima data dari perangkat IoT.

3.4.1 File dan Folder Root Backend

1. `.env`
Menyimpan konfigurasi sensitif seperti nama database, password database, port server, dan secret key JWT.

2. .gitignore
Digunakan untuk mengecualikan file tertentu agar tidak masuk repository.
3. package.json
Berisi daftar library backend dan script server.
4. package-lock.json
Menyimpan detail versi library backend.
5. server.js
Menjalankan server Express, mengatur middleware, menghubungkan seluruh route, serta menginisialisasi koneksi database.

3.4.2 Folder config

1. config.js
Menyimpan konfigurasi database.
2. sequelize.js
Mengatur koneksi ORM Sequelize antara backend dan database.

3.4.3 Folder controllers

1. logincontrollers.js
Menangani autentikasi login dan pembuatan token.
2. registercontroller.js
Menangani pendaftaran user baru.
3. admincontrollers.js
Menangani pengelolaan user dan log sensor.
4. iotcontrollers.js
Menangani penerimaan data dari ESP32 dan penyimpanan data sensor.

3.4.4 Folder middleware

1. auth.js
Memvalidasi token JWT sebelum request diproses.
2. adminOnly.js
Membatasi akses endpoint tertentu agar hanya admin yang dapat mengakses.

3.4.5 Folder models

1. index.js
Menghubungkan seluruh model database.
2. register.js
Mendefinisikan struktur tabel user.

3. sensorlog.js
Mendefinisikan struktur tabel log sensor.

3.4.6 Folder routes

1. login.js
Endpoint login.
2. register.js
Endpoint register.
3. admin.js
Endpoint khusus admin.
4. iot.js
Endpoint pengiriman data sensor.

3.5 Alur Kerja Sistem

1. Sensor membaca suhu dan kelembapan dari lingkungan.
2. ESP32 memproses data hasil pembacaan sensor.
3. ESP32 mengirimkan data ke server backend melalui internet.
4. Backend memvalidasi data dan menyimpannya ke dalam database.
5. Frontend mengambil data dari backend melalui REST API.
6. Backend mengirimkan data dalam format JSON.
7. Dashboard menampilkan data suhu, kelembapan, dan log sensor kepada pengguna secara real-time.

BAB IV – KESIMPULAN

Berdasarkan hasil perancangan, analisis, dan implementasi sistem yang telah dilakukan, dapat disimpulkan bahwa Sistem Perkiraan Cuaca Menggunakan Sensor Temperatur dan Humidity berhasil dirancang dan diimplementasikan dengan baik. Sistem ini memanfaatkan teknologi Internet of Things (IoT) untuk melakukan pemantauan suhu dan kelembapan udara secara real-time melalui sensor yang terhubung dengan perangkat mikrokontroler ESP32.

Data hasil pembacaan sensor dikirimkan ke server backend untuk dilakukan proses validasi dan penyimpanan ke dalam basis data, kemudian disajikan kepada pengguna melalui aplikasi web. Aplikasi web yang dikembangkan mampu menampilkan informasi kondisi cuaca saat ini, data sensor, serta menyediakan fitur pencarian cuaca berdasarkan negara atau kota. Selain itu, sistem juga menerapkan mekanisme autentikasi dan pengelolaan hak akses berbasis peran, di mana admin memiliki kewenangan tambahan untuk mengelola data pengguna, melihat log sensor, serta mengakses informasi prakiraan cuaca lima hari.

Pemodelan sistem menggunakan berbagai diagram UML, seperti use case diagram, activity diagram, sequence diagram, class diagram, dan Entity Relationship Diagram (ERD), membantu dalam menggambarkan alur kerja, struktur sistem, serta interaksi antar komponen secara sistematis dan terstruktur. Arsitektur sistem yang diterapkan dengan pendekatan N-Tier Architecture mendukung pemisahan tanggung jawab antar lapisan sehingga sistem menjadi lebih modular, aman, dan mudah dikembangkan.

Secara keseluruhan, sistem yang dibangun telah memenuhi tujuan penelitian dan dapat dijadikan sebagai sarana pembelajaran dalam penerapan teknologi IoT, pengembangan aplikasi web, serta integrasi perangkat keras dan perangkat lunak. Meskipun demikian, sistem ini masih memiliki keterbatasan, terutama pada jumlah parameter cuaca yang digunakan dan skala pengujian yang terbatas. Oleh karena itu, pengembangan lebih lanjut dapat dilakukan dengan menambahkan sensor lain, meningkatkan akurasi pengukuran, serta memperluas implementasi sistem agar dapat digunakan secara lebih luas dan optimal.

DAFTAR PUSTAKA

- Ashton, K. (2009). That “Internet of Things” Thing. *RFID Journal*, 22(7), 97–114. <https://www.rfidjournal.com/expert-views/that-internet-of-things-thing/73881/>
- Badan Meteorologi, K. dan G. (2023). *Informasi cuaca dan iklim di Indonesia*. Badan Meteorologi, Klimatologi, Dan Geofisika. <https://www.bmkg.go.id/>
- Satria, B. (2022). IoT Monitoring Suhu dan Kelembaban Udara dengan Node MCU ESP8266. *Sudo Jurnal Teknik Informatika*, 1(3), 136–144. <https://doi.org/10.56211/sudo.v1i3.95>

LAMPIRAN

1. Code

Code

Use

Case

@startuml

title Use Case Diagram

title Sistem Informasi Suhu & Cuaca

left to right direction

actor Admin

actor User

actor "IoT Device" as IoT

rectangle "Sistem Informasi Cuaca" {

 usecase "Login" as UC_Login

 usecase "Register" as UC_Register

 usecase "Lihat Cuaca Saat Ini" as UC_Current

 usecase "Lihat Cuaca Negara/Kota" as UC_Country

 usecase "Lihat Forecast 5 Hari" as UC_Forecast

 usecase "Lihat Lokasi Geografis" as UC_Map

 usecase "Lihat Log Sensor" as UC_Log

 usecase "Kelola User" as UC_ManageUser

 usecase "Edit User" as UC_EditUser

 usecase "Hapus User" as UC_DeleteUser

 usecase "Kirim Data Sensor" as UC_Send

}

' ===== RELASI AKTOR =====

Admin --> UC_Current

Admin --> UC_Country

Admin --> UC_Forecast

Admin --> UC_Map

Admin --> UC_Log

Admin --> UC_ManageUser

Admin --> UC_Register

User --> UC_Current

User --> UC_Register

IoT --> UC_Send

'===== SEMUA AKSES USER & ADMIN WAJIB LOGIN =====

UC_Current ..> UC_Login : <<include>>

UC_Country ..> UC_Login : <<include>>

UC_Forecast ..> UC_Login : <<include>>

UC_Map ..> UC_Login : <<include>>

UC_Log ..> UC_Login : <<include>>

UC_ManageUser ..> UC_Login : <<include>>

UC_Register ..> UC_Login : <<extend>>

'===== DETAIL KELOLA USER =====

UC_ManageUser ..> UC_EditUser : <<include>>

UC_ManageUser ..> UC_DeleteUser : <<include>>

Code

Flowchart

@startuml

title Flowchart Sistem Informasi Suhu & Cuaca Berbasis Web

start

:User mengakses website;

if (Sudah login?) then (Ya)

else (Tidak)

:Tampilkan halaman Login / Register;

if (Pilih Register?) then (Ya)

:Input data registrasi;

```
:Simpan data user;
endif

:Input username & password;

:Validasi login;
endif

if (Role = Admin?) then (Admin)

:Masuk Dashboard Admin;

fork

:Monitoring suhu sensor;

:Lihat riwayat log sensor;

fork again

:Cari negara & kota;

:Lihat cuaca saat ini;

:Lihat prakiraan cuaca 5 hari;

:Lihat lokasi geografis;

fork again

:Kelola user (CRUD);

end fork

else (User)

:Masuk Dashboard User;

:Cari negara & kota;

:Lihat cuaca saat ini;

endif

:Logout;

stop
```

@enduml

Activity

Diagram

Register

@startuml

title Activity Diagram - Register

|User/Admin|

start

:Buka halaman register;

:Input data pendaftaran;

|System|

:Validasi data;

if (Data valid?) then (Ya)

 :Simpan data user/admin;

 :Registrasi berhasil;

else (Tidak)

 :Kirim pesan error;

endif

|User/Admin|

stop

@enduml

Activity

Diagram

Login

@startuml

title Activity Diagram - Login

|User / Admin|

start

:Buka halaman login;

:Input username & password;

```

[System]
:Validasi data login;
if (Login valid?) then (Ya)
    :Generate session / token;
    :Arahkan ke dashboard;
else (Tidak)
    :Kirim pesan gagal;
endif
[User / Admin]
stop
@enduml

```

Activity	Diagram	Lihat	Cuaca
@startuml			
title Activity Diagram - Lihat Cuaca Negara/Kota			
[User]			
start			
:Login;			
:Pilih negara & kota;			
[Sistem]			
:Request data cuaca;			
:Ambil data dari API cuaca;			
[User]			
:Lihat cuaca saat ini;			
stop			
@enduml			

Activity **Diagram** **Lihat** **Forecase** **5** **hari** **(hanya** **admin)**
@startuml

title Activity Diagram - Lihat Forecast Cuaca 5 Hari

|Admin|

start

:Login;

:Pilih negara & kota;

|Sistem|

:Request forecast 5 hari;

:Ambil data forecast;

|Admin|

:Lihat hasil forecast;

stop

@enduml

Activity **Diagram** **Kelola** **User** **(admin)**
@startuml

title Activity Diagram - Kelola User

|Admin|

start

:Login;

:Pilih menu kelola user;

if (Aksi?) then (Tambah)

 :Input data user baru;

|Sistem|

:Validasi data;

|Database|

```
:Simpan user;
elseif (Edit)
    :Pilih user;
    :Ubah data;
    |Database|
    :Update data user;
else (Hapus)
    :Pilih user;
    |Database|
    :Hapus data user;
endif
|Admin|
:Kembali ke dashboard;
stop
@enduml
```

Activity	Diagram	Lihat	Log	Sensor	(admin)
@startuml					

title Activity Diagram - Lihat Log Sensor

```
|Admin|
start
:Login;
:Pilih menu log sensor;
|Sistem|
:Ambil data sensor;
|Database|
```

:Query log sensor;

|Sistem|

:Tampilkan riwayat data;

|Admin|

stop

@enduml

Activity	Diagram	Kirim	Data	Sensor	(IOT)
----------	---------	-------	------	--------	-------

@startuml

title Activity Diagram - Kirim Data Sensor

|IoT Device|

start

:Baca sensor temperatur;

|Sistem|

:Terima data sensor;

:Validasi data;

|Database|

:Simpan data sensor;

|IoT Device|

:Terima respon sukses;

stop

@enduml

Sequence	Diagram	(User)
----------	---------	--------

@startuml

title Sequence Diagram

title Login & Monitoring Cuaca

actor User

participant "React Frontend" as React

participant "Backend API" as API

database DB

User -> React : Akses Dashboard

React -> API : Request data

API -> API : Validasi JWT

alt Belum Login

API --> React : 401 Unauthorized

React -> User : Tampilkan Login

User -> React : Input Login

React -> API : POST /login

API -> DB : Validasi User

DB --> API : Valid

API --> React : Token JWT

end

React -> API : Request Data Cuaca

API -> DB : Ambil Data

DB --> API : Data

API --> React : JSON

React -> User : Tampilkan Cuaca

@enduml

Sequence

@startuml

Diagram

(Admin)

title Sequence Diagram

title Aktivitas Admin Sistem Informasi Suhu & Cuaca

actor Admin

participant "React Frontend" as React

participant "Backend API" as API

participant "Weather API\n(OpenWeather)" as Weather

database DB

Admin -> React : Akses Admin Dashboard

React -> API : Request Admin Data

API -> API : Validasi JWT & Role Admin

alt Tidak Terautentikasi / Bukan Admin

API --> React : 403 Forbidden

React -> Admin : Redirect ke Login

Admin -> React : Input Login

React -> API : POST /login

API -> DB : Validasi Admin

DB --> API : Admin Valid

API --> React : Token JWT

end

== Monitoring Cuaca ==

React -> API : Request Cuaca Saat Ini

API -> Weather : Request Current Weather

Weather --> API : Data Cuaca

API --> React : JSON Cuaca

React -> API : Request Forecast 5 Hari

API -> Weather : Request Forecast

Weather --> API : Data Forecast

API --> React : JSON Forecast

== Lokasi Geografis ==

React -> API : Request Koordinat Kota/Negara

API -> Weather : Request Geo Location

Weather --> API : Data Lokasi

API --> React : JSON Lokasi

== Log Sensor ==

React -> API : Request Log Sensor

API -> DB : Query Sensor Log

DB --> API : Data Log

API --> React : JSON Log

== Kelola User ==

React -> API : Request Data User

API -> DB : Ambil Data User

DB --> API : Data User

API --> React : JSON User

alt Edit User

React -> API : PUT /user/{id}

API -> DB : Update User

DB --> API : Success

API --> React : OK

end

alt Hapus User

React -> API : DELETE /user/{id}

API -> DB : Delete User

DB --> API : Success

API --> React : OK

end

@enduml

Sequence

Diagram

(IOT)

@startuml

title Sequence Diagram

title Pengiriman Data Sensor

participant "IoT Device" as IoT

participant "Backend API" as API

database DB

IoT -> IoT : Baca Sensor

IoT -> API : Kirim Data

API -> API : Validasi API Key

API -> DB : Simpan Sensor Log

DB --> API : OK

API --> IoT : Success

@enduml

Class

Diagram

@startuml

title Class Diagram

title Sistem Informasi Suhu Cuaca Berbasis Web

skinparam classAttributeIconSize 0

skinparam style strictuml

'=====

' USER & ROLE

'=====

class User {

- userId : int

- username : string

- password : string

- role : string

+ login()

+ register()

+ logout()

}

class Admin {

+ manageUser()

+ viewSensorLogs()

}

Admin --|> User

'=====

' AUTHENTICATION

'=====

class AuthService {

+ authenticate(username, password)

+ generateToken(user)


```
}
```

```
User --> AuthService : login
```

```
' =====
```

```
' WEATHER (CUACA GLOBAL)
```

```
' =====
```

```
class WeatherService {
```

```
    + getCurrentWeather(city)
```

```
    + getForecast(city)
```

```
    + getGeoLocation(city)
```

```
}
```

```
class WeatherData {
```

```
    - country : string
```

```
    - city : string
```

```
    - temperature : float
```

```
    - humidity : float
```

```
    - forecast5Days : string
```

```
    - latitude : float
```

```
    - longitude : float
```

```
}
```

```
User --> WeatherService : request
```

```
WeatherService --> WeatherData : produces
```

```
' =====
```

```
' SENSOR & IOT
```

```
' =====
```

```

class IoTDevice {
    - deviceId : int
    - apiKey : string
    + sendData()
}

class SensorService {
    + receiveSensorData()
    + getSensorLogs()
}

class SensorData {
    - dataId : int
    - temperature : float
    - timestamp : datetime
}

IoTDevice --> SensorService : send
SensorService --> SensorData : create
IoTDevice "1" --> "0..*" SensorData
Admin --> SensorService : view

' =====

' DATABASE

' =====

class Database {
    + connect()
    + query()

```

}

SensorService --> Database : store/read

WeatherService --> Database : store/read

AuthService --> Database : validate

@enduml

Entity	Relationship	Diagram	(ERD)
--------	--------------	---------	-------

@startuml

title ERD Sistem Monitoring IoT (Users & Sensor Logs)

' Pengaturan tampilan agar lebih profesional

skinparam linetype ortho

skinparam shadowing false

skinparam class {

 BackgroundColor White

 ArrowColor #2688d4

 BorderColor #2688d4

}

entity "Users" as User {

 * **id_user** : INT <<PK>> <<AI>>

 --

 nama_lengkap : VARCHAR(100)

 email : VARCHAR(100) <<UNIQUE>>

 password : VARCHAR(255)

 role : ENUM('admin', 'warga')

 createdAt : DATETIME

 updatedAt : DATETIME

```

}
entity "Sensor_Log" as Sensor {
    * **id** : INT <<PK>> <<AI>>
    --
    suhu : FLOAT
    kelembaban : FLOAT
    cahaya : INT
    createdAt : DATETIME
    updatedAt : DATETIME
}
' Relasi: User memantau log sensor
User ||--o{ Sensor : "memantau"
@enduml

```

Arsitektur

Diagram

```
@startuml
```

```
title Arsitektur Sistem Informasi Suhu & Cuaca Berbasis Web
```

```
skinparam componentStyle rectangle
```

```
skinparam shadowing false
```

```
actor "Admin / User" as User
```

```
package "Client Side" {
```

```
    component "Web Browser"
```

```
    component "React.js\nFrontend UI" as React
```

```
}
```

```
package "Server Side" {
```

```
    component "Node.js\nExpress.js API" as API
```

```

    component "Auth Service\nJWT" as Auth
    component "Weather Service\n(OpenWeather API)" as Weather
    component "Sensor Service" as SensorService
}

package "IoT Layer" {
    component "Temperature Sensor\n(DHT11/DHT22)" as Sensor
    component "IoT Device\n(ESP32)" as IoT
}

package "Database Layer" {
    database "MySQL" as DB
}

```

User --> React : Akses Web

React --> API : Request REST API

API --> Auth : Validasi Login

API --> Weather : Request Data Cuaca

API --> SensorService : Proses Data Sensor

SensorService --> DB : Simpan / Ambil Data

Weather --> API : Data Cuaca

API --> React : Response JSON

React --> User : Tampilkan Data

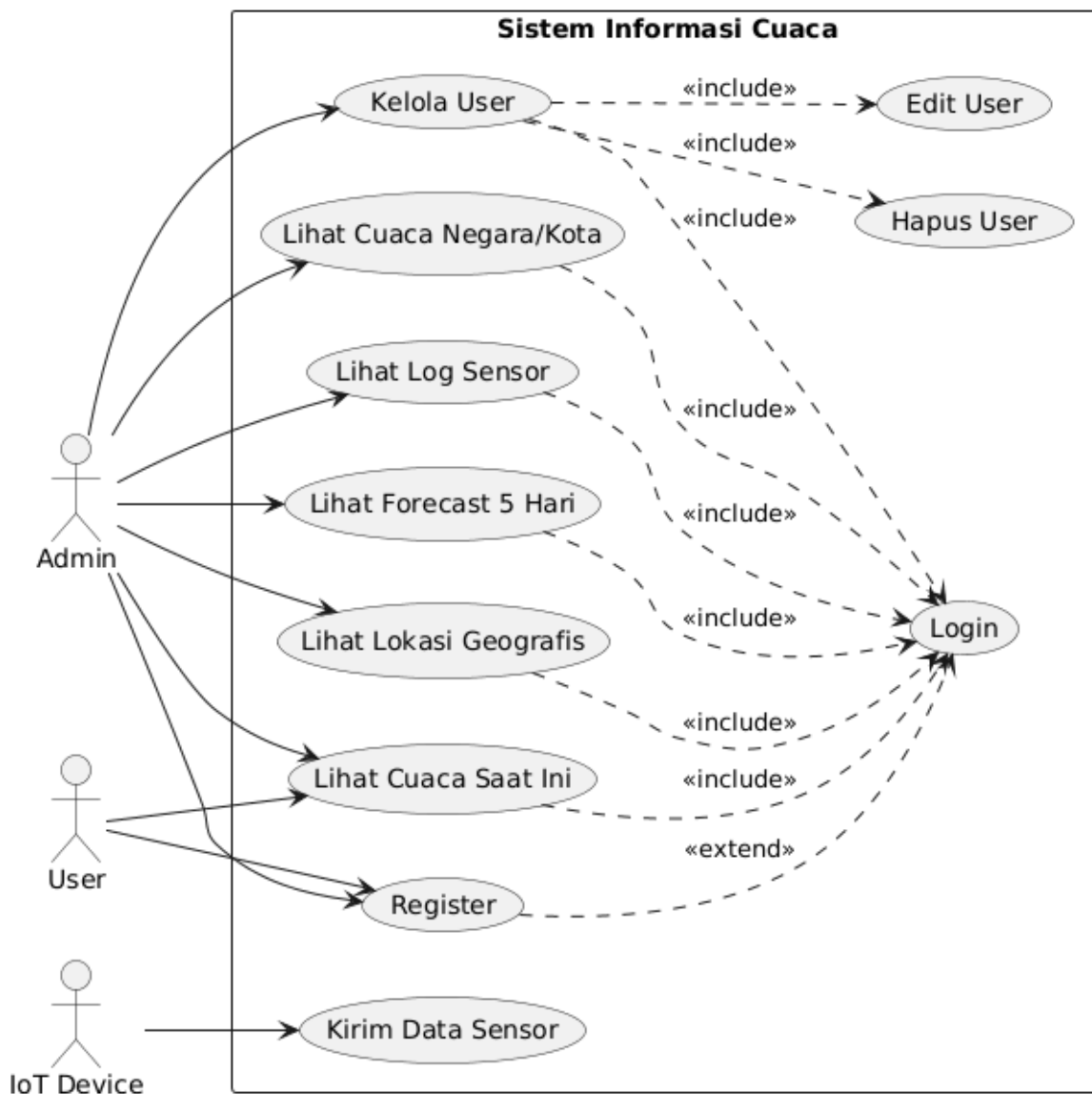
IoT --> API : Kirim Data Sensor

API --> DB : Simpan Log Sensor

@enduml

2. Gambar

Sistem Informasi Suhu & Cuaca



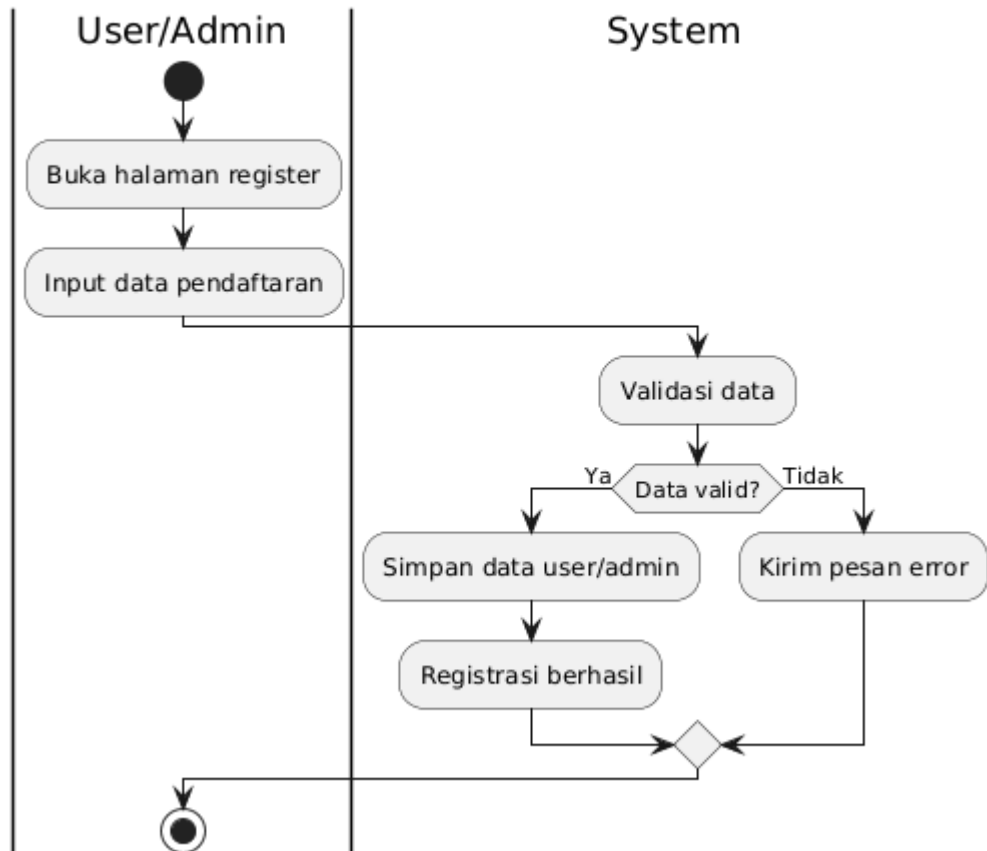
Gambar 1 Use Case Diagram Sistem Informasi Suhu dan Cuaca

```

graph TD
    Start(( )) --> A[User mengakses website]
    A --> B{Sudah login?}
    B -- Ya --> D[Input username & password]
    B -- Tidak --> C[Tampilkan halaman Login / Register]
    C --> E{Pilih Register?}
    E -- Ya --> F[Input data registrasi]
    F --> G[Simpan data user]
    G --> H{ }
    E -- Tidak --> H
    H --> D
    D --> I[Validasi login]
    I --> J{ }
    J -- Admin --> K[Masuk Dashboard Admin]
    J -- User --> L[Masuk Dashboard User]
    J -- "Role = Admin?" --> J
    K --> M[Monitoring suhu sensor]
    K --> N[Cari negara & kota]
    K --> O[Kelola user (CRUD)]
    M --> P[Lihat riwayat log sensor]
    N --> Q[Lihat cuaca saat ini]
    Q --> R[Lihat prakiraan cuaca 5 hari]
    R --> S[Lihat lokasi geografis]
    L --> T[Cari negara & kota]
    T --> U[Lihat cuaca saat ini]
    P --> V{ }
    Q --> V
    R --> V
    S --> V
    T --> V
    U --> V
    V --> W[Logout]
    W --> End(( ))
  
```

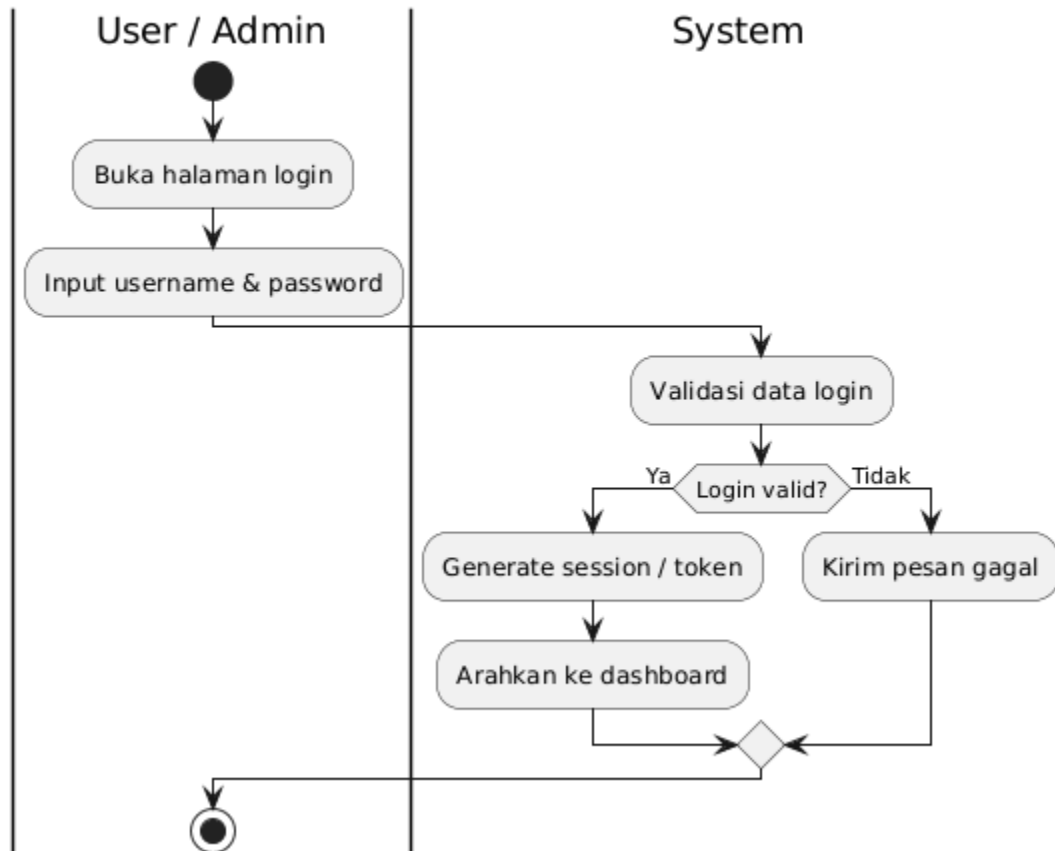
Gambar 2 Flowchart Diagram Sistem Informasi Suhu dan Cuaca

Activity Diagram - Register



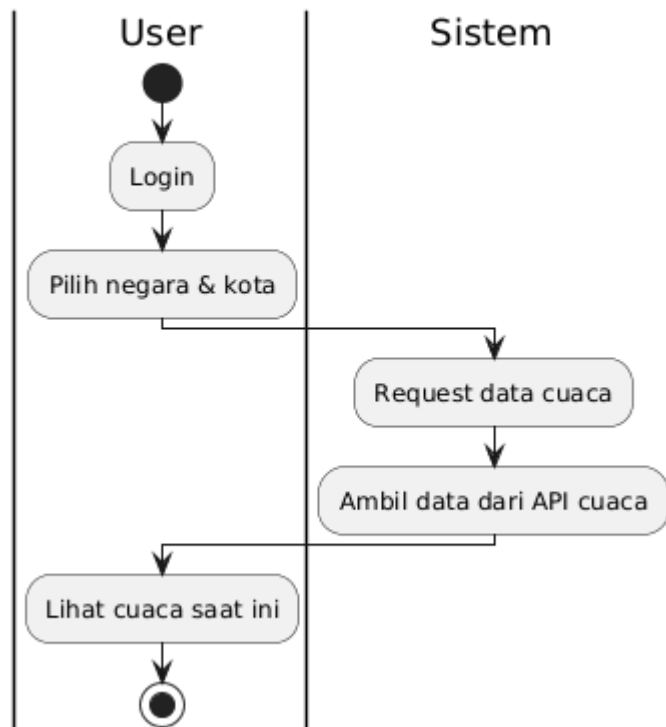
Gambar 3 Activity Diagram Register

Activity Diagram - Login



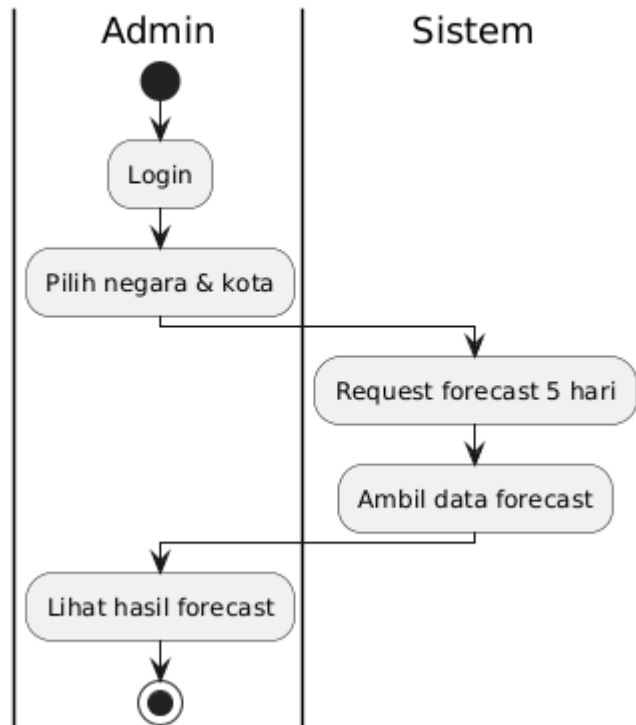
Gambar 4 Activity Diagram Login

Activity Diagram - Lihat Cuaca Negara/Kota

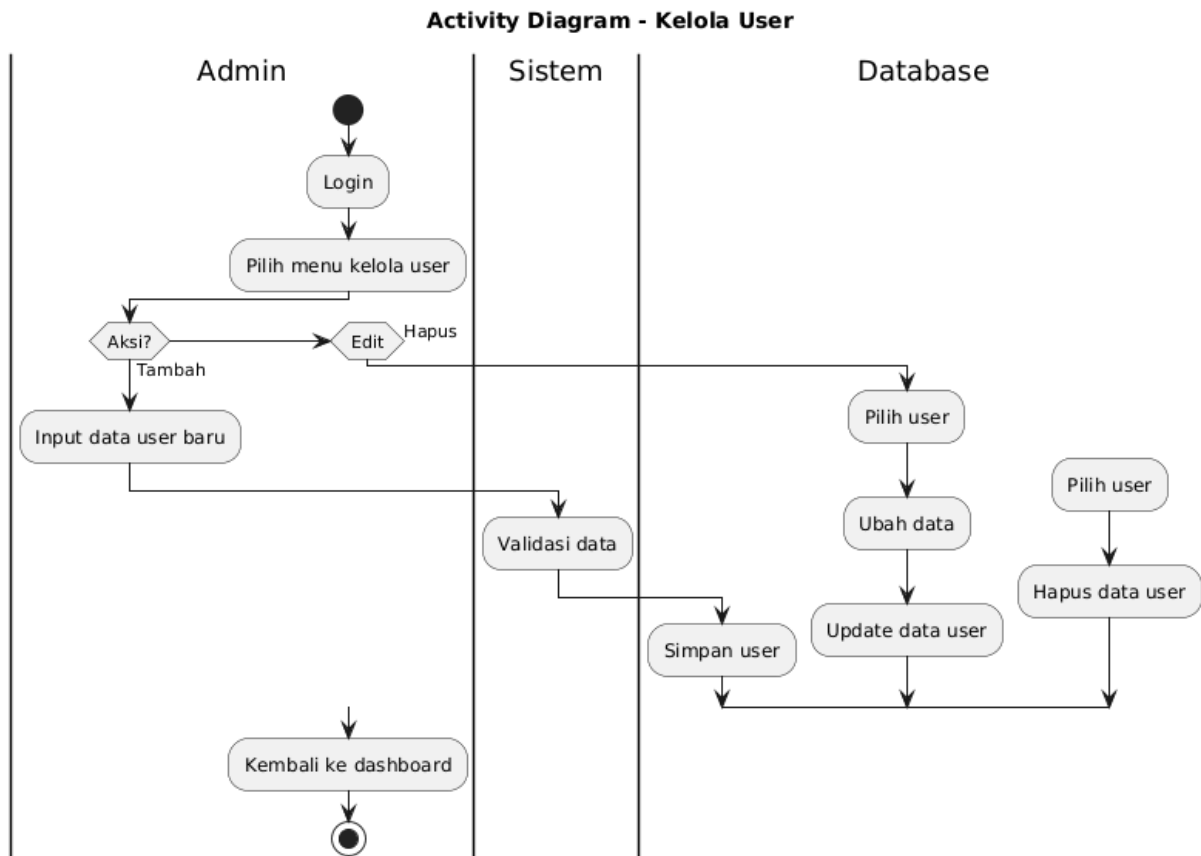


Gambar 5 Activity Diagram Lihat Cuaca Negara/Kota (User)

Activity Diagram - Lihat Forecast Cuaca 5 Hari

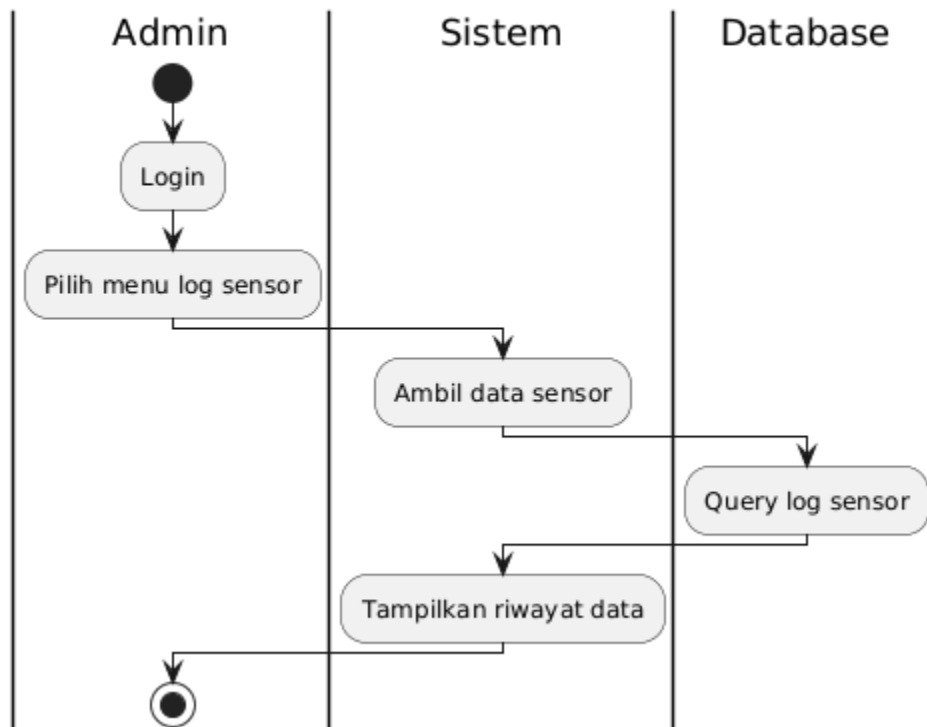


Gambar 6 Activity Diagram Lihat Forecast Cuaca 5 Hari (Admin)

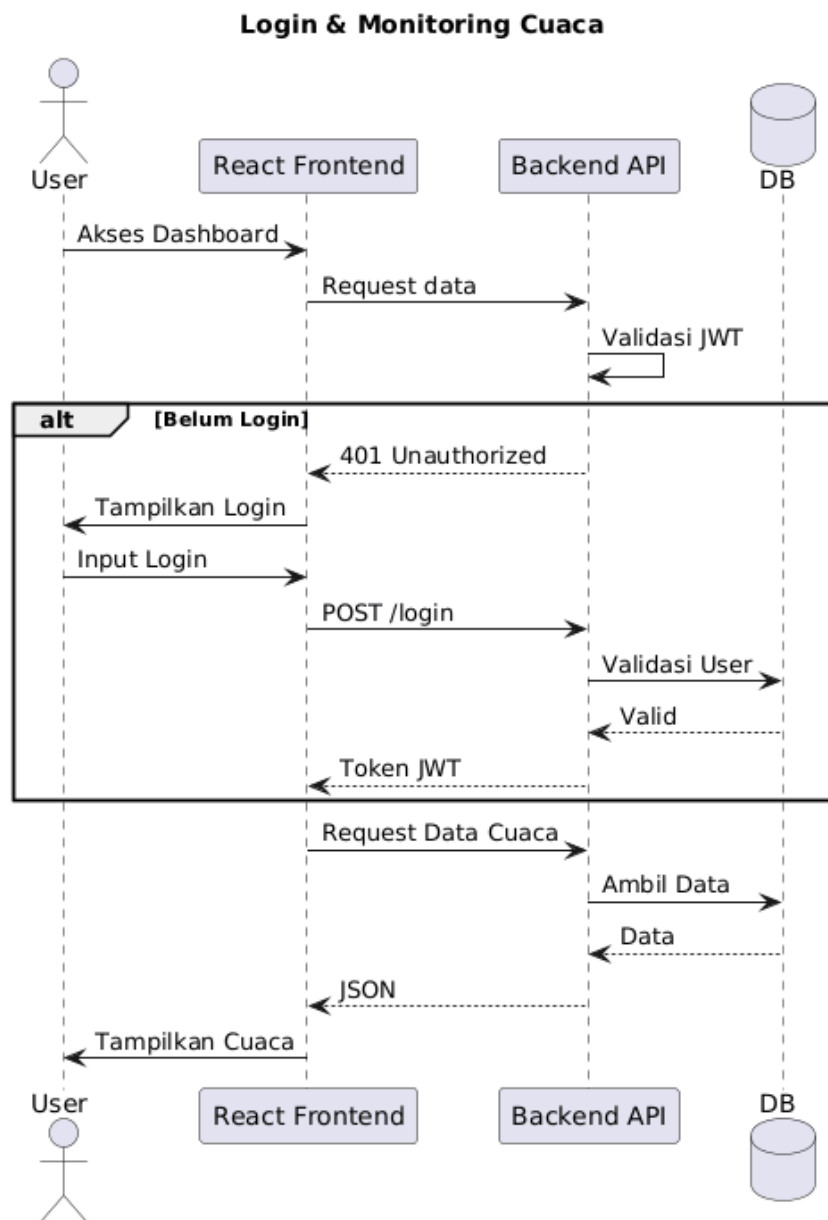


Gambar 7 Activity Diagram Kelola User (Admin)

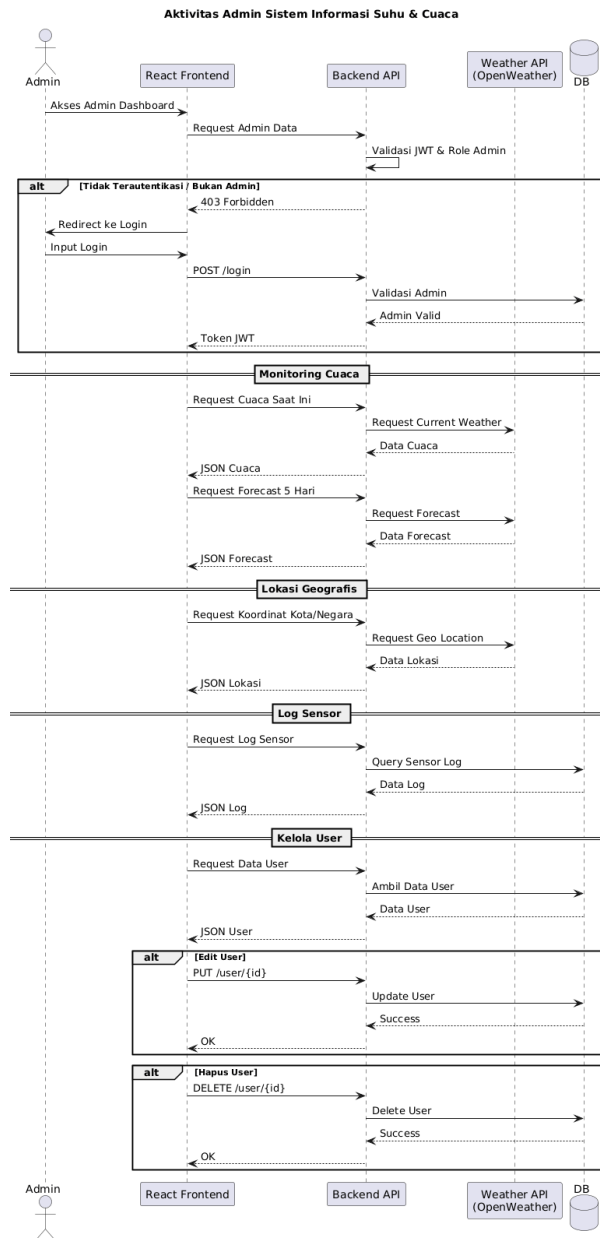
Activity Diagram - Lihat Log Sensor



Gambar 9 Activity Diagram Kirim Data Sensor (IOT)

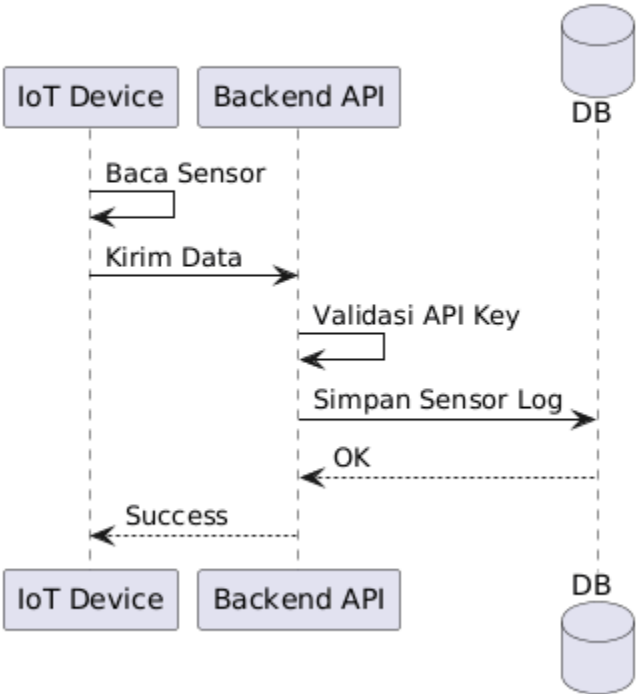


Gambar 10 Sequence Diagram (User)



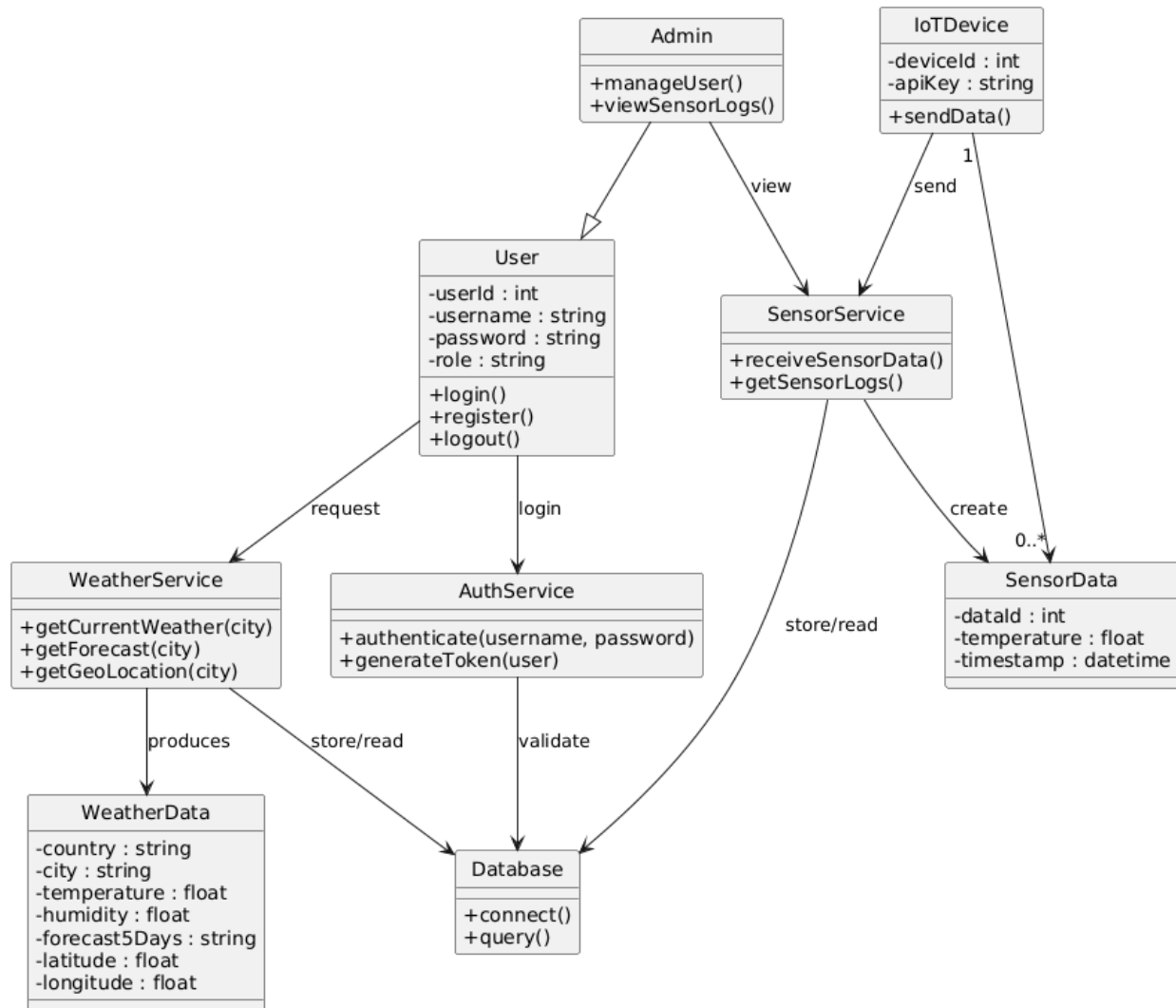
Gambar 11 Sequence Diagram (Admin)

Pengiriman Data Sensor



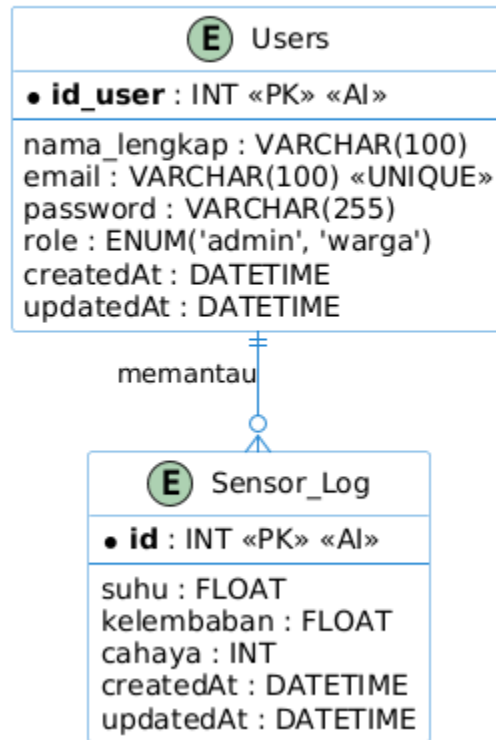
Gambar 12 Sequence Diagram Pengiriman Data Sensor (IOT)

Sistem Informasi Suhu Cuaca Berbasis Web



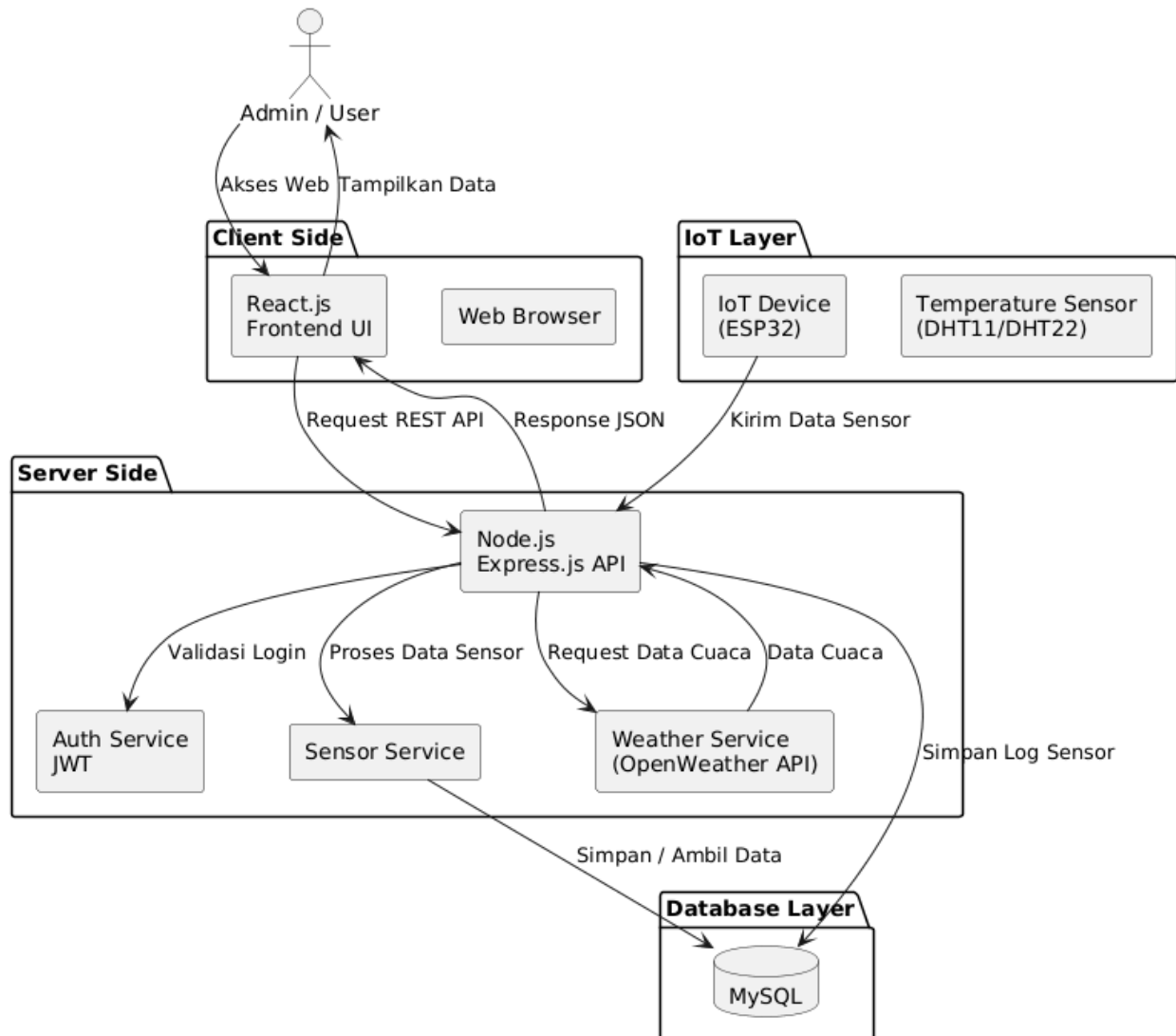
Gambar 14 Class Diagram Sistem Informasi Suhu dan Cuaca

ERD Sistem Monitoring IoT (Users & Sensor Logs)



Gambar 15 ERD Sistem Monitoring IoT (User dan Sensor Logs)

Arsitektur Sistem Informasi Suhu & Cuaca Berbasis Web



Gambar 16 Arsitektur Diagram Sistem Informasi Suhu dan Cuaca



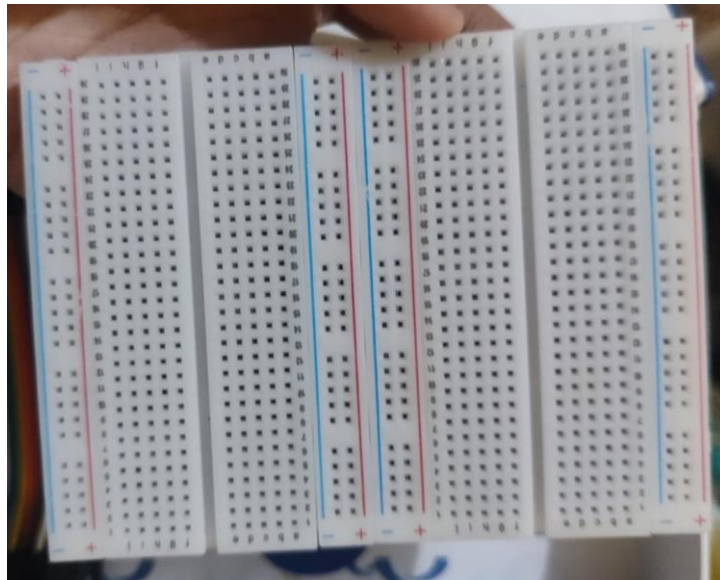
Gambar 17 alat ESP32



Gambar 18 Sensor Temperatur dan Humidity



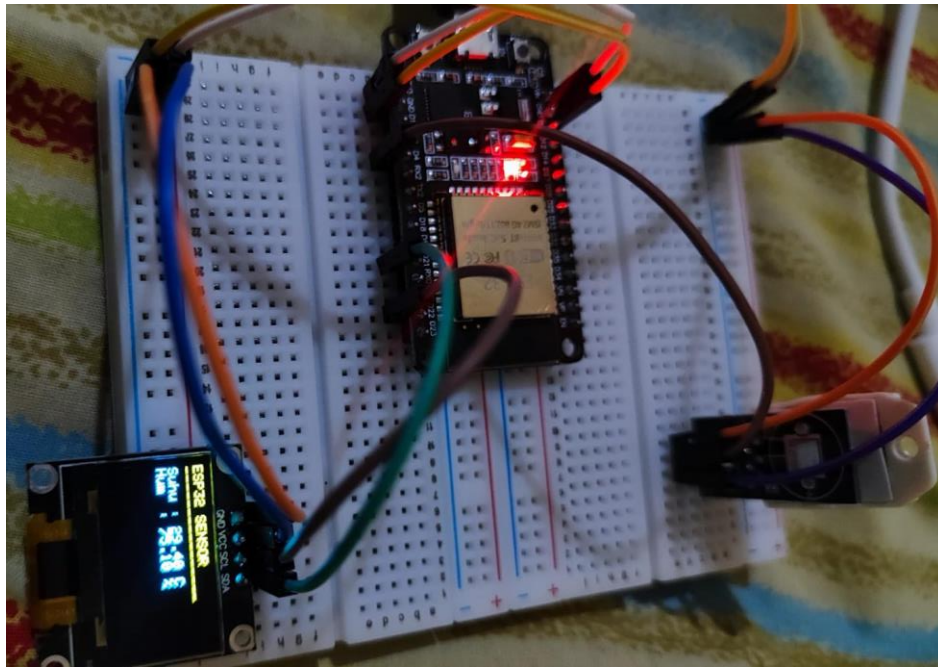
Gambar 19 Kabel Jumper



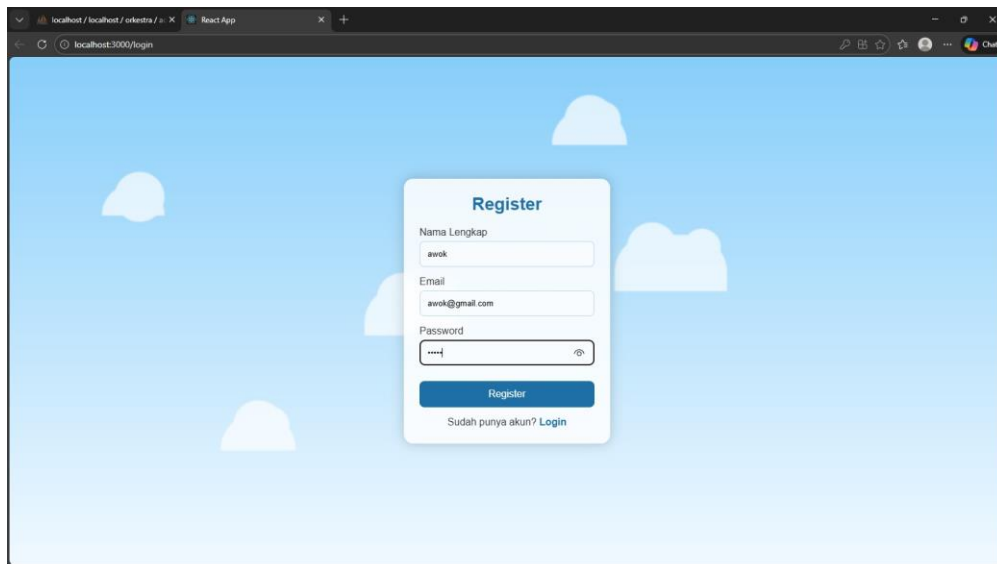
Gambar 20 Breadboard



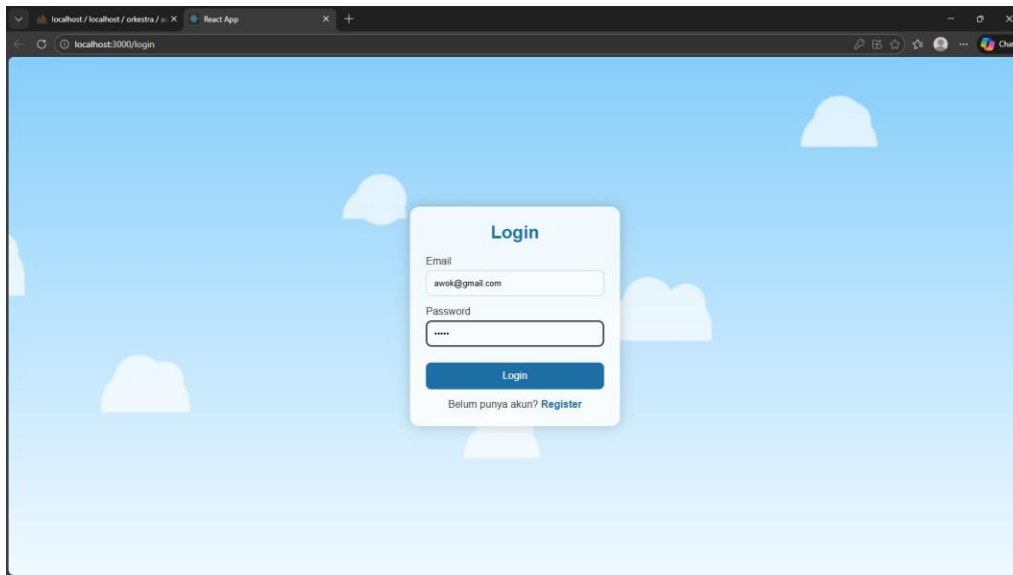
Gambar 21 Kabel USB



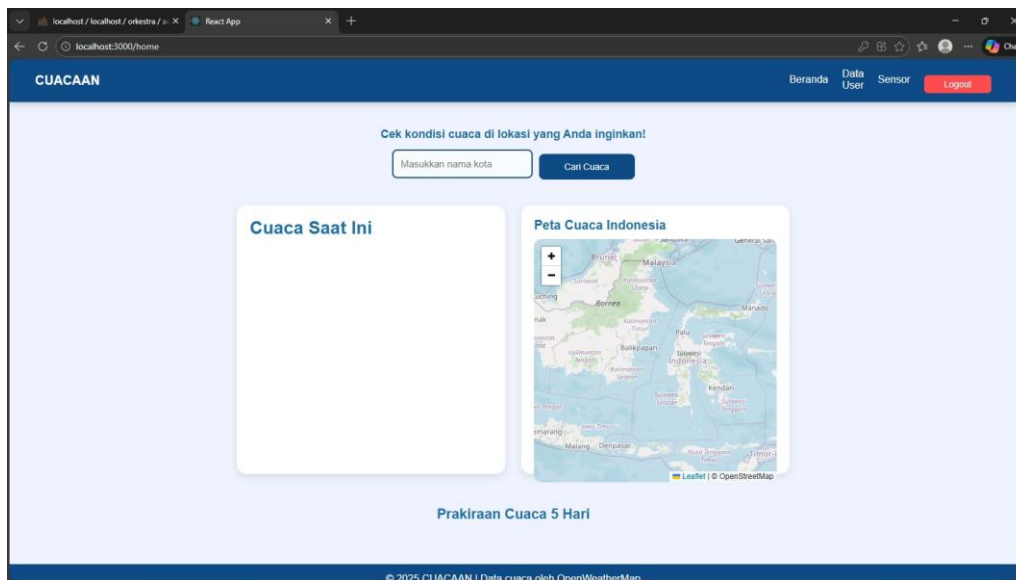
Gambar 22 Gambar Alat



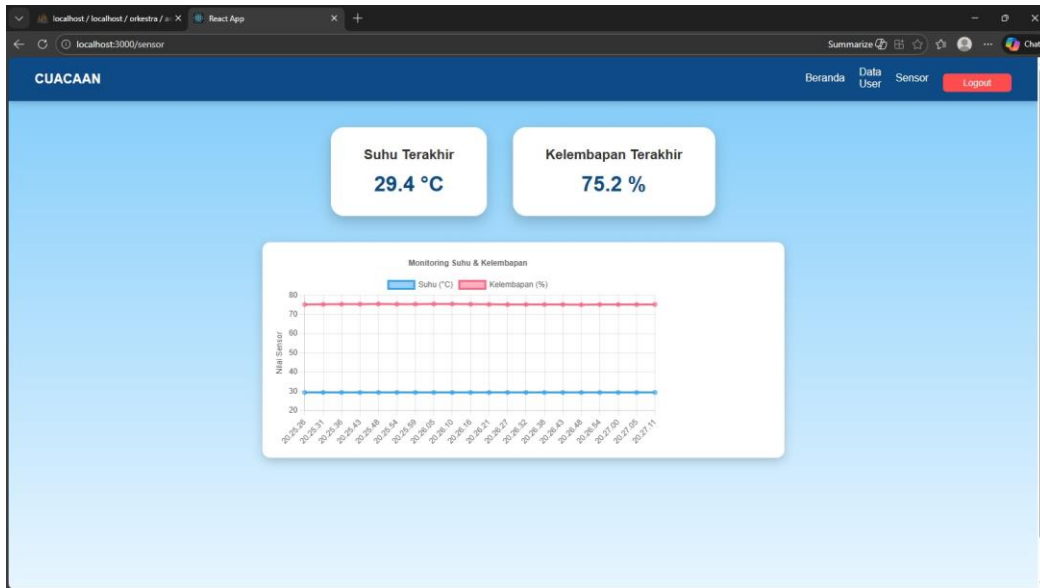
Gambar 23 Tampilan Registrasi User

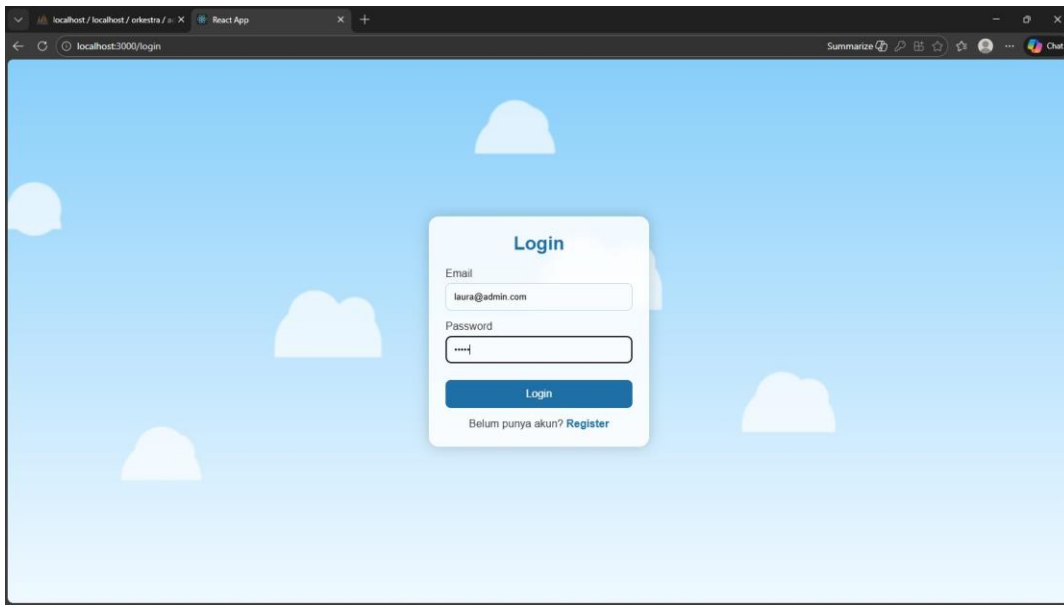


Gambar 24 Tampilan Login User

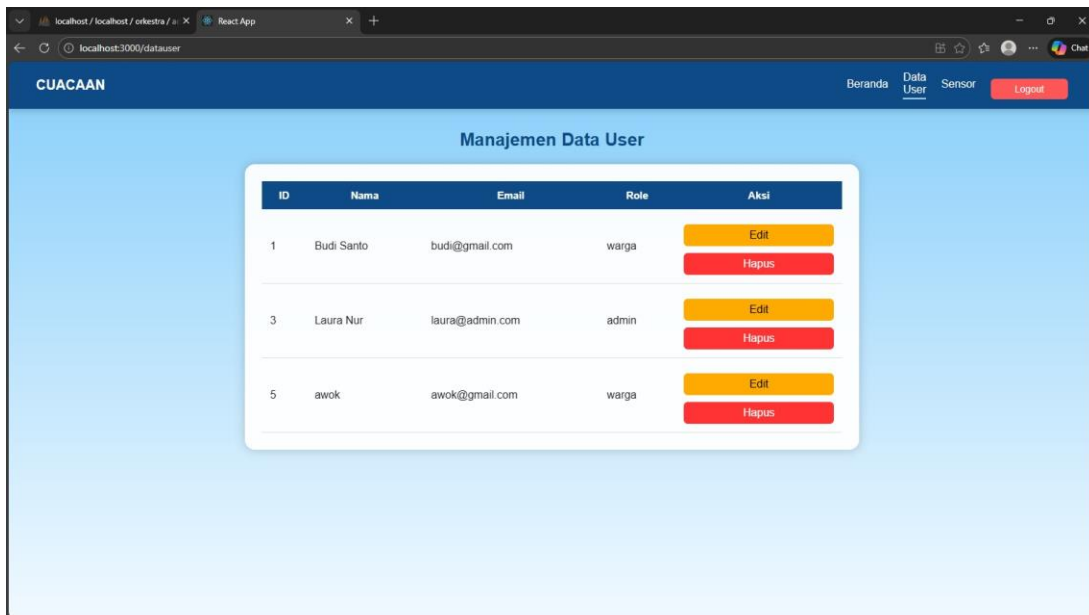


Gambar 25 Tampilan Homepage User

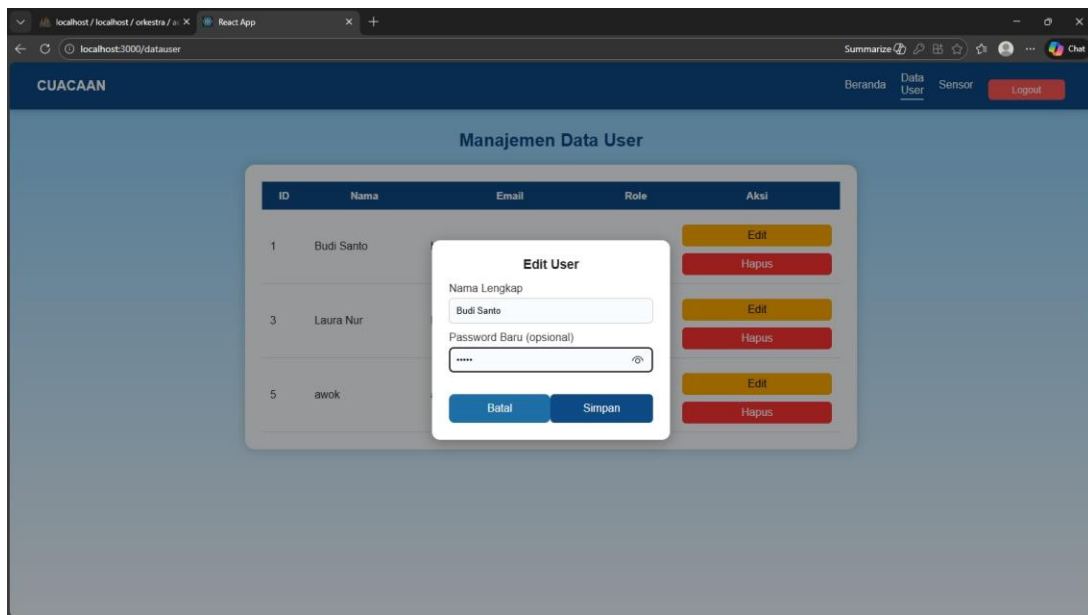




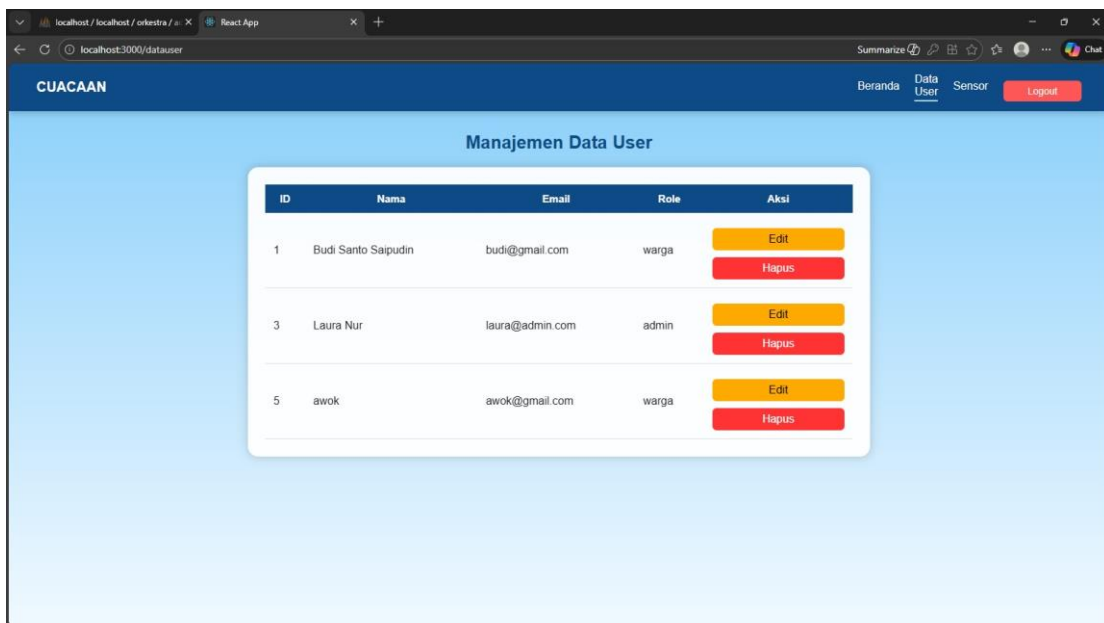
Gambar 28 Tampilan Login Admin



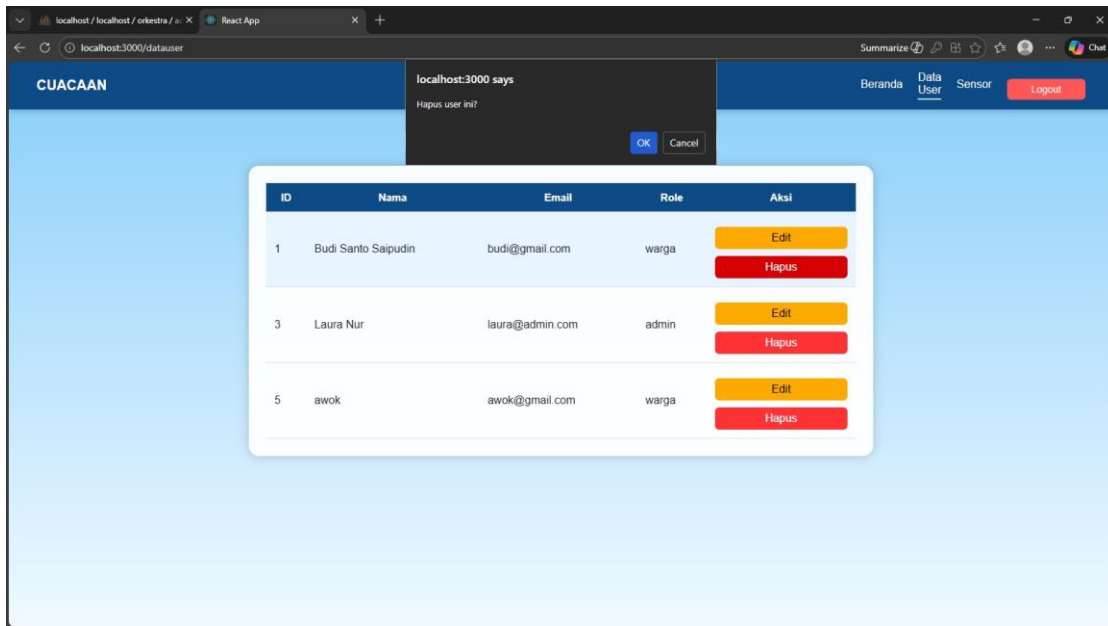
Gambar 29 Tampilan Halaman Manajemen Data User



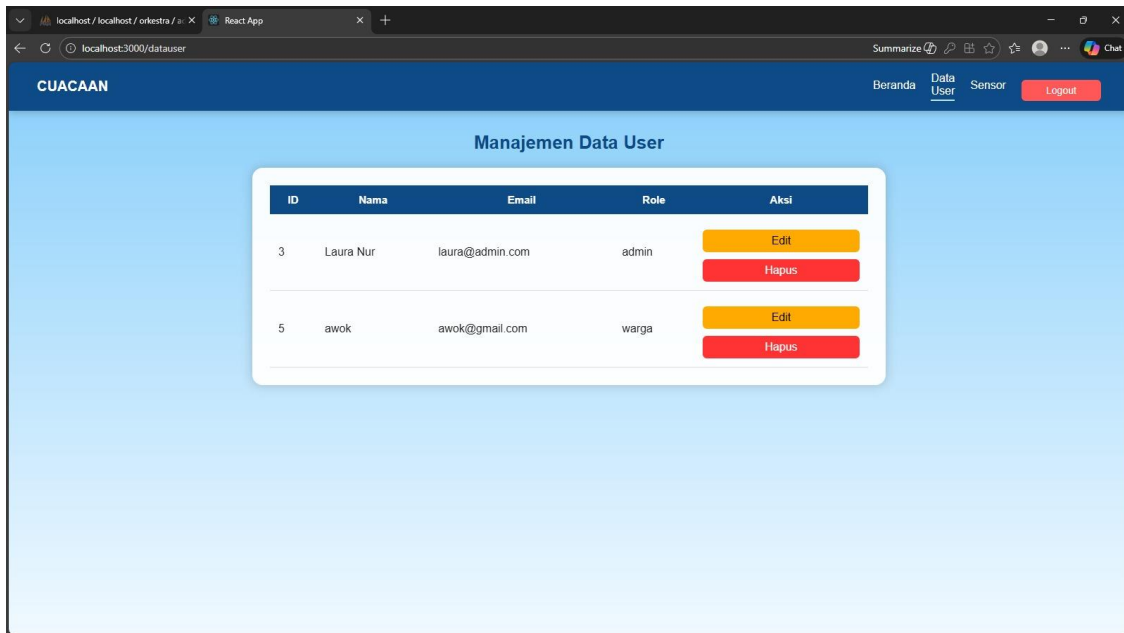
Gambar 30 Tampilan Halaman Edit User



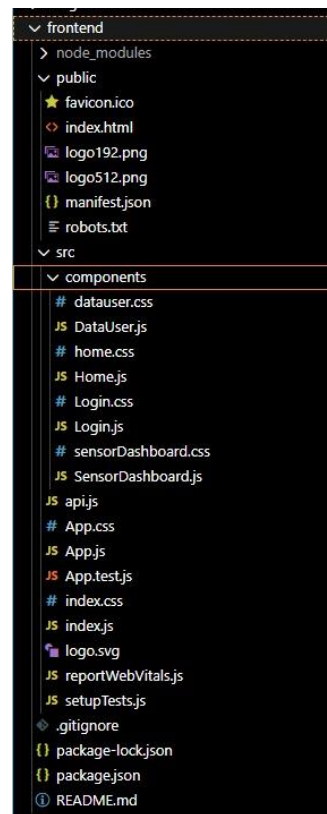
Gambar 31 Tampilan Setelah Edit



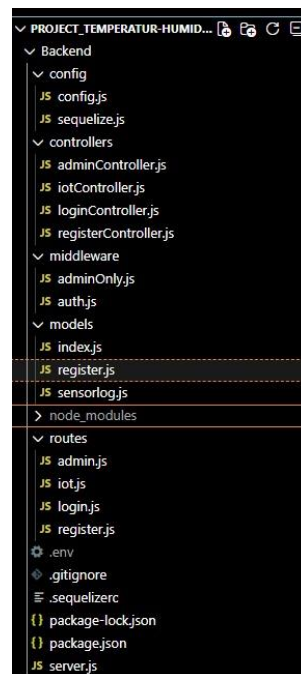
Gambar 32 Tampilan Hapus User



Gambar 33 Tampilan Setelah Hapus



Gambar 34 Tampilan Folder Frontend



Gambar 35 Tampilan Folder Backend

